Red Neuronal y Deep Learning

Nombre: Erick Mejia

Redes Neuronales y Deep Learning

VC04_V05: Deep Learning y Deep vision

In [1]:
```
# SOLO PARA USO EN GOOGLE COLABORATORY
# Para conectar el notebook con la cuenta de gdrive
from google.colab import drive
drive.mount('/content/drive/', force_remount=True)

BASE_FOLDER = '/content/drive/My Drive/archivo/' # Se debe garantizar que la carpet
```

```
Mounted at /content/drive/
```

INTRODUCCIÓN A LAS CONVOLUTIONAL NEURAL NETWORKS: MNIST DATASET

- Cargando el conjunto de datos

In [2]:
```
import tensorflow as tf
mnist = tf.keras.datasets.mnist
(x_train, y_train), (x_test, y_test) = mnist.load_data()
#print(x_train.shape)
#print(y_train.shape)
#print(x_test.shape)
#print(y_test.shape)
```

- Acondicionando el conjunto de datos

In [3]:
```
# Pre-procesado obligatorio cuando trabajo con redes neuronales
from tensorflow.keras.utils import to_categorical
from sklearn.model_selection import train_test_split
from tensorflow.keras.backend import expand_dims

x_train, x_te = x_train / 255.0, x_test / 255.0 #Cambio al rango 0-1 -> Disminuyo C
#¿Que pasa si empleo labels con etiquetas número entero?
print(y_train[0])
#y_train = to_categorical(y_train, num_classes=10) #One-hot encoding para minimizar
#y_te = to_categorical(y_test, num_classes=10)
x_tr, x_val, y_tr, y_val = train_test_split(x_train, y_train, test_size=0.1, random
#Expandir dimensiones porque en CNN tengo que especificar el número de canales
print(x_tr.shape)
x_tr = expand_dims(x_tr, axis=3)
x_val = expand_dims(x_val, axis=3)
x_te = expand_dims(x_te, axis=3)
print(x_tr.shape)
```

```
5
(54000, 28, 28)
(54000, 28, 28, 1)
```

- Creando la topología de Red Neuronal (CNN) y entrenándola

In [4]:
```
# Construccion de una red CNN
from tensorflow.keras.models import Sequential
from tensorflow.keras import layers
# Red feedforward API secuencial
convnet = Sequential()

# BASE MODEL
convnet.add(layers.Conv2D(32,(3,3),input_shape=(28,28,1),activation='relu', padding
convnet.add(layers.MaxPooling2D((2,2)))

convnet.add(layers.Conv2D(64,(3,3),activation='relu',padding='same'))
convnet.add(layers.MaxPooling2D((2,2)))

convnet.add(layers.Conv2D(64,(3,3),activation='relu', padding='same'))

#TOP MODEL
convnet.add(layers.Flatten())
convnet.add(layers.Dense(64,activation='relu'))
convnet.add(layers.Dense(10,activation='softmax'))
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:
113: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first layer in
the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

In [5]:
```
convnet.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 28, 28, 32)	320
max_pooling2d (MaxPooling2D)	(None, 14, 14, 32)	0
conv2d_1 (Conv2D)	(None, 14, 14, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 7, 7, 64)	0
conv2d_2 (Conv2D)	(None, 7, 7, 64)	36,928
flatten (Flatten)	(None, 3136)	0
dense (Dense)	(None, 64)	200,768
dense_1 (Dense)	(None, 10)	650

Total params: 257,162 (1004.54 KB)

Trainable params: 257,162 (1004.54 KB)

Non-trainable params: 0 (0.00 B)

In [6]:
```
convnet.compile(optimizer='adam',
               loss='sparse_categorical_crossentropy', #If labels are integers
               #loss='categorical_crossentropy', #If labels are one-hot encoded
               metrics=['accuracy'])
```

In [9]:
```
H = convnet.fit(x_tr, y_tr, epochs=5, batch_size=128, validation_data=(x_val, y_val
```

```
Epoch 1/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 9s 12ms/step - accuracy: 0.9338 - loss: 0.2114 - val_ac
curacy: 0.9800 - val_loss: 0.0636
Epoch 2/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.9830 - loss: 0.0538 - val_acc
uracy: 0.9880 - val_loss: 0.0416
Epoch 3/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 0.9884 - loss: 0.0364 - val_acc
uracy: 0.9913 - val_loss: 0.0337
Epoch 4/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.9916 - loss: 0.0277 - val_acc
uracy: 0.9907 - val_loss: 0.0326
Epoch 5/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.9934 - loss: 0.0206 - val_acc
uracy: 0.9912 - val_loss: 0.0312
```

- Observando el proceso de entrenamiento para tomar decisiones

In [10]:
```
import matplotlib.pyplot as plt
import numpy as np
# Muestro gráfica de accuracy y losses
plt.style.use("ggplot")
plt.figure()
plt.plot(np.arange(0, 5), H.history["loss"], label="train_loss")
plt.plot(np.arange(0, 5), H.history["val_loss"], label="val_loss")
plt.plot(np.arange(0, 5), H.history["accuracy"], label="train_acc")
plt.plot(np.arange(0, 5), H.history["val_accuracy"], label="val_acc")
plt.title("Training Loss and Accuracy")
plt.xlabel("Epoch #")
plt.ylabel("Loss/Accuracy")
plt.legend()
```

Out[10]:
```
<matplotlib.legend.Legend at 0x7a50426632f0>
```

- Probando el conjunto de datos en el subset de test y evaluando el performance del modelo

In [11]:
```
from sklearn.metrics import classification_report
# Evaluando el modelo de predicción con las imágenes de test
print("[INFO]: Evaluando red neuronal...")
predictions = convnet.predict(x_te, batch_size=128)
#print(y_te[0])
```

```
#print(predictions[0])
print(classification_report(y_test, predictions.argmax(axis=1)))
```

```
[INFO]: Evaluando red neuronal...
79/79 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step
              precision    recall  f1-score   support

           0       0.99      1.00      0.99       980
           1       1.00      0.99      0.99      1135
           2       1.00      0.99      1.00      1032
           3       0.99      1.00      0.99      1010
           4       0.99      0.99      0.99       982
           5       0.99      0.99      0.99       892
           6       0.99      0.99      0.99       958
           7       0.99      1.00      0.99      1028
           8       0.99      0.99      0.99       974
           9       0.99      0.98      0.98      1009

    accuracy                           0.99     10000
   macro avg       0.99      0.99      0.99     10000
weighted avg       0.99      0.99      0.99     10000
```

¿POR QUE CONVOLUTIONAL NEURAL NETWORKS?: CIFAR DATASET

- Cargando el conjunto de datos y acondicionándolo

In [12]:
```
# Importando el set de datos CIFAR10
from tensorflow.keras.datasets import cifar10
from sklearn.preprocessing import LabelBinarizer
print("[INFO]: Loading CIFAR-10 data...")
((trainX, trainY), (testX, testY)) = cifar10.load_data()
trainX = trainX.astype("float") / 255.0
testX = testX.astype("float") / 255.0
labelNames = ["Avión", "Automóvil", "Pájaro", "Gato", "Ciervo", "Perro", "Rana", "C
print(trainX.shape)
print(trainY.shape)
# Por si es necesario convertir a one-hot encoding
#lb = LabelBinarizer()
#trainY = lb.fit_transform(trainY)
#testY = lb.transform(testY)
```

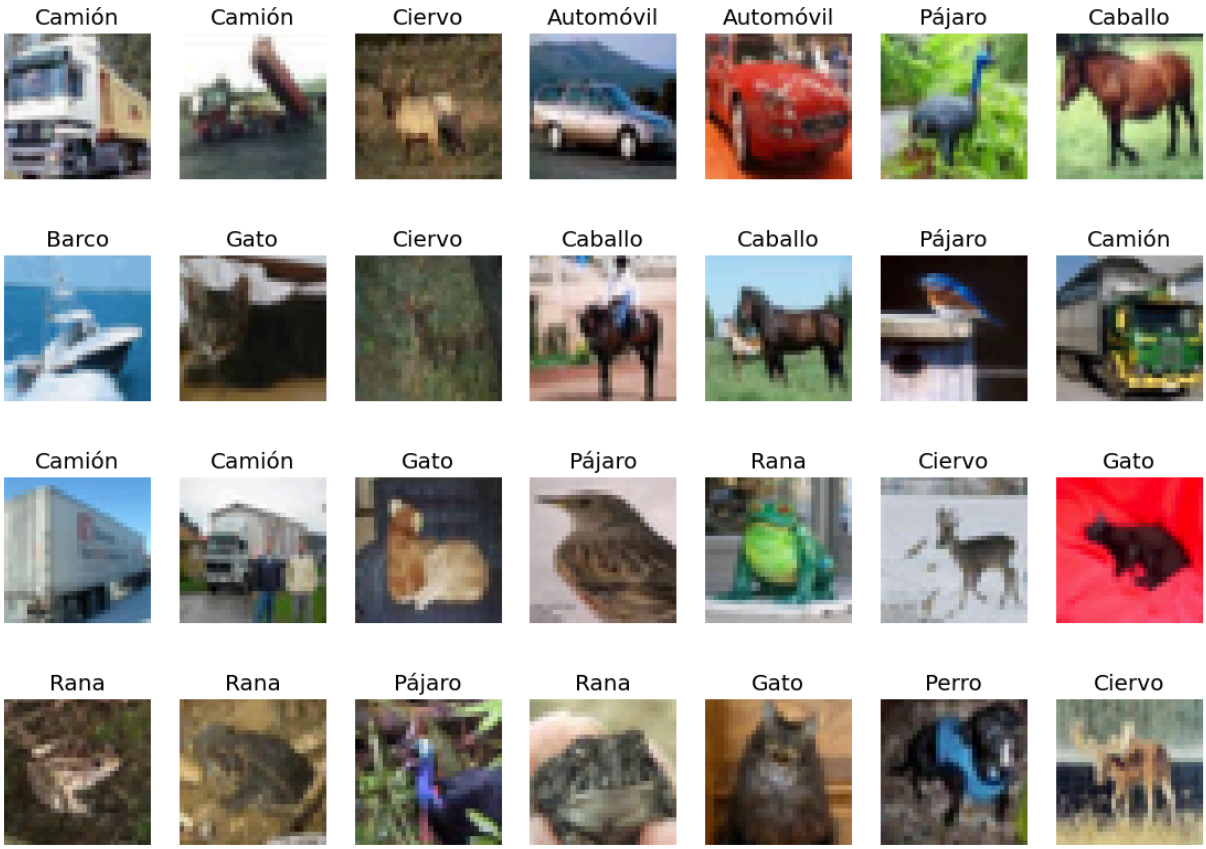
```
[INFO]: Loading CIFAR-10 data...
(50000, 32, 32, 3)
(50000, 1)
```

In [13]:
```
print(testX.shape)
print(testY.shape)
```

```
(10000, 32, 32, 3)
(10000, 1)
```

- Inspeccionando el conjunto de datos

In [14]:
```
import matplotlib.pyplot as plt
fig = plt.figure(figsize=(14,10))
for n in range(1, 29):
    fig.add_subplot(4, 7, n)
    img = trainX[n]
    plt.imshow(img)
    plt.title(labelNames[trainY[n][0]])
    plt.axis('off')
```



- Creando la topología de red neuronal y entrenándola: MLP

In [15]:
```
# Imports necesarios
import numpy as np
from sklearn.metrics import classification_report
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten, Dropout
from tensorflow.keras.optimizers import SGD
import matplotlib.pyplot as plt

# Arquitectura de red
# Definimos el modo API Sequential
model = Sequential() #(X)
model.add(Flatten()) #(X)
# Primera capa oculta
model.add(Dense(2048, input_shape=(32*32*3,), activation="relu")) #(X)
#model.add(Dropout(0.5))
# Segunda capa oculta
model.add(Dense(1024, activation="relu")) #(X)
#model.add(Dropout(0.5))
```

```python
# Tercera capa oculta
model.add(Dense(512, activation="relu")) #(X)
#model.add(Dropout(0.5))
# Cuarta capa oculta
model.add(Dense(128, activation="relu")) #(X)
#model.add(Dropout(0.5))
# Quinta capa oculta
model.add(Dense(32, activation="relu")) #(X)
# Capa de salida
model.add(Dense(10, activation="softmax")) #(X)


# Compilamos el modelo y entrenamos
print("[INFO]: Entrenando red neuronal...")
# Compilamos el modelo
model.compile(loss="sparse_categorical_crossentropy", optimizer=SGD(0.01), metrics=
# model.compile(loss="categorical_crossentropy", optimizer=SGD(0.01), metrics=["acc
# Entrenamos el perceptrón multicapa
H = model.fit(trainX, trainY, validation_split=0.2, epochs=50, batch_size=32) #(X)

# Evaluamos con las muestras de test
print("[INFO]: Evaluando modelo...")
# Efectuamos predicciones
predictions = model.predict(testX, batch_size=32) #(X)
# Obtenemos el report
print(classification_report(testY, predictions.argmax(axis=1), target_names=labelNa
# print(classification_report(testY.argmax(axis=1), predictions.argmax(axis=1), tar

# Mostramos gráfica de accuracy y losses
plt.style.use("ggplot")
plt.figure()
plt.plot(np.arange(0, 50), H.history["loss"], label="train_loss")
plt.plot(np.arange(0, 50), H.history["val_loss"], label="val_loss")
plt.plot(np.arange(0, 50), H.history["accuracy"], label="train_acc")
plt.plot(np.arange(0, 50), H.history["val_accuracy"], label="val_acc")
plt.title("Training Loss and Accuracy")
plt.xlabel("Epoch #")
plt.ylabel("Loss/Accuracy")
plt.legend()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarn
ing: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequen
tial models, prefer using an `Input(shape)` object as the first layer in the model i
nstead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

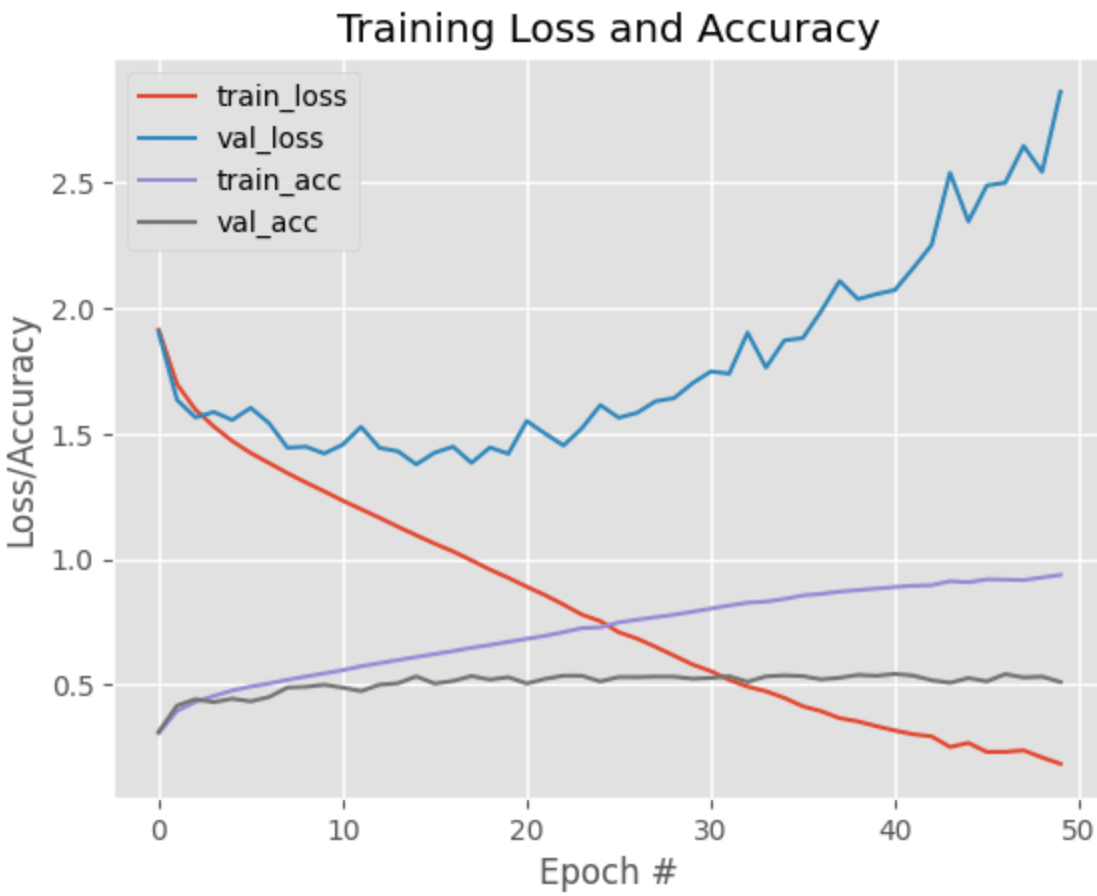
```
[INFO]: Entrenando red neuronal...
Epoch 1/50
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 9s 6ms/step - accuracy: 0.3067 - loss: 1.9142 - val_a
ccuracy: 0.3104 - val_loss: 1.9054
Epoch 2/50
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.3940 - loss: 1.6974 - val_a
ccuracy: 0.4156 - val_loss: 1.6345
Epoch 3/50
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.4304 - loss: 1.5957 - val_a
ccuracy: 0.4409 - val_loss: 1.5648
Epoch 4/50
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.4538 - loss: 1.5286 - val_a
ccuracy: 0.4293 - val_loss: 1.5864
Epoch 5/50
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.4757 - loss: 1.4705 - val_a
ccuracy: 0.4431 - val_loss: 1.5545
Epoch 6/50
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.4911 - loss: 1.4229 - val_a
ccuracy: 0.4324 - val_loss: 1.6031
Epoch 7/50
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 6s 5ms/step - accuracy: 0.5037 - loss: 1.3830 - val_a
ccuracy: 0.4498 - val_loss: 1.5418
Epoch 8/50
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.5183 - loss: 1.3427 - val_a
ccuracy: 0.4874 - val_loss: 1.4439
Epoch 9/50
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.5314 - loss: 1.3053 - val_a
ccuracy: 0.4905 - val_loss: 1.4481
Epoch 10/50
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 6s 5ms/step - accuracy: 0.5441 - loss: 1.2702 - val_a
ccuracy: 0.4985 - val_loss: 1.4209
Epoch 11/50
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.5567 - loss: 1.2329 - val_a
ccuracy: 0.4872 - val_loss: 1.4562
Epoch 12/50
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.5724 - loss: 1.1988 - val_a
ccuracy: 0.4732 - val_loss: 1.5272
Epoch 13/50
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.5846 - loss: 1.1646 - val_a
ccuracy: 0.4989 - val_loss: 1.4433
Epoch 14/50
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.5970 - loss: 1.1290 - val_a
ccuracy: 0.5045 - val_loss: 1.4300
Epoch 15/50
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.6091 - loss: 1.0950 - val_a
ccuracy: 0.5320 - val_loss: 1.3776
Epoch 16/50
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.6213 - loss: 1.0613 - val_a
ccuracy: 0.5034 - val_loss: 1.4244
Epoch 17/50
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.6327 - loss: 1.0303 - val_a
ccuracy: 0.5139 - val_loss: 1.4482
Epoch 18/50
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 6s 5ms/step - accuracy: 0.6460 - loss: 0.9949 - val_a
ccuracy: 0.5334 - val_loss: 1.3836
Epoch 19/50
```

```
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.6573 - loss: 0.9580 - val_a
ccuracy: 0.5191 - val_loss: 1.4449
Epoch 20/50
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.6698 - loss: 0.9252 - val_a
ccuracy: 0.5288 - val_loss: 1.4201
Epoch 21/50
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 6s 5ms/step - accuracy: 0.6817 - loss: 0.8896 - val_a
ccuracy: 0.5038 - val_loss: 1.5506
Epoch 22/50
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.6938 - loss: 0.8560 - val_a
ccuracy: 0.5220 - val_loss: 1.5002
Epoch 23/50
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.7085 - loss: 0.8184 - val_a
ccuracy: 0.5349 - val_loss: 1.4529
Epoch 24/50
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.7246 - loss: 0.7783 - val_a
ccuracy: 0.5346 - val_loss: 1.5217
Epoch 25/50
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.7277 - loss: 0.7524 - val_a
ccuracy: 0.5118 - val_loss: 1.6139
Epoch 26/50
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 6s 5ms/step - accuracy: 0.7472 - loss: 0.7082 - val_a
ccuracy: 0.5296 - val_loss: 1.5637
Epoch 27/50
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.7582 - loss: 0.6827 - val_a
ccuracy: 0.5292 - val_loss: 1.5832
Epoch 28/50
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.7681 - loss: 0.6499 - val_a
ccuracy: 0.5310 - val_loss: 1.6293
Epoch 29/50
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 6s 5ms/step - accuracy: 0.7781 - loss: 0.6150 - val_a
ccuracy: 0.5308 - val_loss: 1.6416
Epoch 30/50
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.7899 - loss: 0.5797 - val_a
ccuracy: 0.5227 - val_loss: 1.7006
Epoch 31/50
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.8022 - loss: 0.5530 - val_a
ccuracy: 0.5259 - val_loss: 1.7480
Epoch 32/50
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 6s 5ms/step - accuracy: 0.8147 - loss: 0.5182 - val_a
ccuracy: 0.5325 - val_loss: 1.7395
Epoch 33/50
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.8260 - loss: 0.4911 - val_a
ccuracy: 0.5106 - val_loss: 1.9026
Epoch 34/50
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 6s 5ms/step - accuracy: 0.8303 - loss: 0.4728 - val_a
ccuracy: 0.5320 - val_loss: 1.7638
Epoch 35/50
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.8408 - loss: 0.4462 - val_a
ccuracy: 0.5359 - val_loss: 1.8720
Epoch 36/50
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.8548 - loss: 0.4129 - val_a
ccuracy: 0.5329 - val_loss: 1.8812
Epoch 37/50
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 6s 5ms/step - accuracy: 0.8611 - loss: 0.3931 - val_a
ccuracy: 0.5201 - val_loss: 1.9888
```

```
Epoch 38/50
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 10s 4ms/step - accuracy: 0.8702 - loss: 0.3655 - val_
accuracy: 0.5265 - val_loss: 2.1089
Epoch 39/50
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 10s 4ms/step - accuracy: 0.8760 - loss: 0.3525 - val_
accuracy: 0.5380 - val_loss: 2.0363
Epoch 40/50
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 6s 5ms/step - accuracy: 0.8821 - loss: 0.3333 - val_a
ccuracy: 0.5342 - val_loss: 2.0565
Epoch 41/50
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.8885 - loss: 0.3160 - val_a
ccuracy: 0.5419 - val_loss: 2.0728
Epoch 42/50
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.8935 - loss: 0.3013 - val_a
ccuracy: 0.5350 - val_loss: 2.1605
Epoch 43/50
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 6s 5ms/step - accuracy: 0.8955 - loss: 0.2928 - val_a
ccuracy: 0.5167 - val_loss: 2.2526
Epoch 44/50
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.9116 - loss: 0.2505 - val_a
ccuracy: 0.5066 - val_loss: 2.5405
Epoch 45/50
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.9075 - loss: 0.2665 - val_a
ccuracy: 0.5254 - val_loss: 2.3462
Epoch 46/50
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.9189 - loss: 0.2307 - val_a
ccuracy: 0.5124 - val_loss: 2.4898
Epoch 47/50
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.9178 - loss: 0.2313 - val_a
ccuracy: 0.5411 - val_loss: 2.5008
Epoch 48/50
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 6s 5ms/step - accuracy: 0.9160 - loss: 0.2369 - val_a
ccuracy: 0.5277 - val_loss: 2.6474
Epoch 49/50
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.9264 - loss: 0.2087 - val_a
ccuracy: 0.5311 - val_loss: 2.5444
Epoch 50/50
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.9366 - loss: 0.1833 - val_a
ccuracy: 0.5098 - val_loss: 2.8633
[INFO]: Evaluando modelo...
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step
              precision    recall  f1-score   support

       Avión       0.65      0.57      0.60      1000
   Automóvil       0.67      0.57      0.62      1000
      Pájaro       0.39      0.49      0.44      1000
        Gato       0.31      0.41      0.35      1000
      Ciervo       0.51      0.32      0.39      1000
       Perro       0.37      0.50      0.42      1000
        Rana       0.52      0.63      0.57      1000
     Caballo       0.80      0.34      0.48      1000
       Barco       0.62      0.68      0.65      1000
      Camión       0.59      0.57      0.58      1000

    accuracy                           0.51     10000
   macro avg       0.54      0.51      0.51     10000
```

```
weighted avg       0.54      0.51      0.51     10000
```

Out[15]: <matplotlib.legend.Legend at 0x7a50280ae870>



- Creando la topología de red neuronal y entrenándola: CNN

In [16]:
```
# Import the necessary packages
import numpy as np
from tensorflow.keras import backend as K
from tensorflow.keras.layers import Input, Conv2D, Activation, Flatten, Dense, Drop
from tensorflow.keras.models import Model
from tensorflow.keras.optimizers import SGD, Adam
from sklearn.metrics import classification_report
import matplotlib.pyplot as plt
from google.colab import drive

n_epochs=50
#########################################
###### Definimos la arquitectura ########
#########################################
#BASE MODEL
# Definimos entradas
inputs = Input(shape=(trainX.shape[1], trainX.shape[2], trainX.shape[3]))

# Primer set de capas CONV => RELU => CONV => RELU => POOL
x1 = Conv2D(32, (3, 3), padding="same", activation="relu")(inputs)
x1 = BatchNormalization()(x1)
```

```
x1 = Conv2D(32, (3, 3), padding="same", activation="relu")(x1)
x1 = BatchNormalization()(x1)
x1 = MaxPooling2D(pool_size=(2, 2))(x1)
x1 = Dropout(0.25)(x1)

# Segundo set de capas CONV => RELU => CONV => RELU => POOL
x2 = Conv2D(64, (3, 3), padding="same", activation="relu")(x1) #(X)
x2 = BatchNormalization()(x2) #(X)
x2 = Conv2D(64, (3, 3), padding="same", activation="relu")(x2) #(X)
x2 = BatchNormalization()(x2) #(X)
x2 = MaxPooling2D(pool_size=(2, 2))(x2) #(X)
x2 = Dropout(0.25)(x2) #(X)

# Tercer set de capas CONV => RELU => CONV => RELU => POOL
x3 = Conv2D(256, (3, 3), padding="same", activation="relu")(x2) #(X)
x3 = BatchNormalization()(x3) #(X)
x3 = Conv2D(256, (3, 3), padding="same", activation="relu")(x3) #(X)
x3 = BatchNormalization()(x3) #(X)
x3 = MaxPooling2D(pool_size=(2, 2))(x3) #(X)
x3 = Dropout(0.25)(x3) #(X)

# TOP MODEL
# Primer (y único) set de capas FC => RELU
xfc = Flatten()(x3) #(X)
xfc = Dense(512, activation="relu")(xfc) #(X)
xfc = BatchNormalization()(xfc) #(X)
xfc = Dropout(0.5)(xfc) #(X)
# Clasificador softmax
predictions = Dense(10, activation="softmax")(xfc) #(X)

# Unimos las entradas y el modelo mediante la función Model con parámetros inputs y
model_cnn = Model(inputs=inputs, outputs=predictions) #(X)

# Compilar el modelo
print("[INFO]: Compilando el modelo...")
model_cnn.compile(loss="sparse_categorical_crossentropy", optimizer=Adam(learning_r

# Entrenamiento de la red
print("[INFO]: Entrenando la red...")
H = model_cnn.fit(trainX, trainY, validation_split=0.2, batch_size=128, epochs=n_ep

# Almaceno el modelo en Drive
# Montamos la unidad de Drive
drive.mount('/content/drive') #(X)
# Almacenamos el modelo empleando la función mdoel.save de Keras
model_cnn.save(BASE_FOLDER+"deepCNN_CIFAR10.h5") #(X)

# Evaluación del modelo
print("[INFO]: Evaluando el modelo...")
# Efectuamos la predicción (empleamos el mismo valor de batch_size que en training)
predictions = model_cnn.predict(testX, batch_size=128) #(X)
# Sacamos el report para test
print(classification_report(testY, predictions.argmax(axis=1), target_names=labelNa

# Gráficas
plt.style.use("ggplot")
```

```
plt.figure()
plt.plot(np.arange(0, n_epochs), H.history["loss"], label="train_loss")
plt.plot(np.arange(0, n_epochs), H.history["val_loss"], label="val_loss")
plt.plot(np.arange(0, n_epochs), H.history["accuracy"], label="train_acc")
plt.plot(np.arange(0, n_epochs), H.history["val_accuracy"], label="val_acc")
plt.title("Training Loss and Accuracy")
plt.xlabel("Epoch #")
plt.ylabel("Loss/Accuracy")
plt.legend()
plt.show()
```

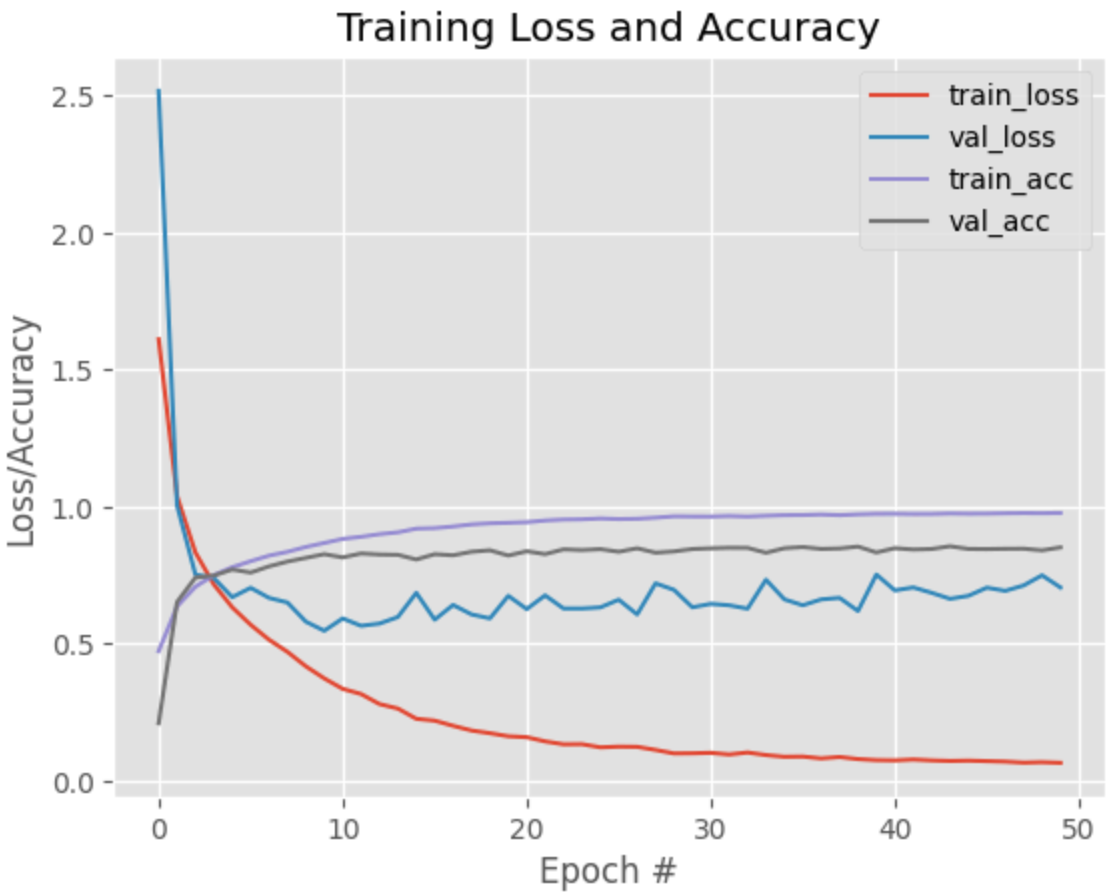
```
[INFO]: Compilando el modelo...
[INFO]: Entrenando la red...
Epoch 1/50
313/313 ━━━━━━━━━━━━━━━━━━━━ 34s 65ms/step - accuracy: 0.4731 - loss: 1.6110 - val_a
ccuracy: 0.2099 - val_loss: 2.5173
Epoch 2/50
313/313 ━━━━━━━━━━━━━━━━━━━━ 7s 22ms/step - accuracy: 0.6364 - loss: 1.0384 - val_ac
curacy: 0.6522 - val_loss: 1.0050
Epoch 3/50
313/313 ━━━━━━━━━━━━━━━━━━━━ 7s 22ms/step - accuracy: 0.7081 - loss: 0.8329 - val_ac
curacy: 0.7415 - val_loss: 0.7522
Epoch 4/50
313/313 ━━━━━━━━━━━━━━━━━━━━ 7s 22ms/step - accuracy: 0.7503 - loss: 0.7163 - val_ac
curacy: 0.7486 - val_loss: 0.7398
Epoch 5/50
313/313 ━━━━━━━━━━━━━━━━━━━━ 7s 22ms/step - accuracy: 0.7791 - loss: 0.6328 - val_ac
curacy: 0.7709 - val_loss: 0.6699
Epoch 6/50
313/313 ━━━━━━━━━━━━━━━━━━━━ 7s 22ms/step - accuracy: 0.8011 - loss: 0.5692 - val_ac
curacy: 0.7597 - val_loss: 0.7040
Epoch 7/50
313/313 ━━━━━━━━━━━━━━━━━━━━ 7s 22ms/step - accuracy: 0.8215 - loss: 0.5147 - val_ac
curacy: 0.7823 - val_loss: 0.6673
Epoch 8/50
313/313 ━━━━━━━━━━━━━━━━━━━━ 7s 22ms/step - accuracy: 0.8353 - loss: 0.4703 - val_ac
curacy: 0.7998 - val_loss: 0.6504
Epoch 9/50
313/313 ━━━━━━━━━━━━━━━━━━━━ 7s 22ms/step - accuracy: 0.8531 - loss: 0.4176 - val_ac
curacy: 0.8133 - val_loss: 0.5805
Epoch 10/50
313/313 ━━━━━━━━━━━━━━━━━━━━ 7s 22ms/step - accuracy: 0.8677 - loss: 0.3735 - val_ac
curacy: 0.8269 - val_loss: 0.5473
Epoch 11/50
313/313 ━━━━━━━━━━━━━━━━━━━━ 7s 22ms/step - accuracy: 0.8825 - loss: 0.3355 - val_ac
curacy: 0.8150 - val_loss: 0.5924
Epoch 12/50
313/313 ━━━━━━━━━━━━━━━━━━━━ 7s 22ms/step - accuracy: 0.8902 - loss: 0.3165 - val_ac
curacy: 0.8293 - val_loss: 0.5654
Epoch 13/50
313/313 ━━━━━━━━━━━━━━━━━━━━ 7s 22ms/step - accuracy: 0.8997 - loss: 0.2799 - val_ac
curacy: 0.8255 - val_loss: 0.5736
Epoch 14/50
313/313 ━━━━━━━━━━━━━━━━━━━━ 7s 23ms/step - accuracy: 0.9065 - loss: 0.2636 - val_ac
curacy: 0.8244 - val_loss: 0.5975
Epoch 15/50
313/313 ━━━━━━━━━━━━━━━━━━━━ 7s 22ms/step - accuracy: 0.9200 - loss: 0.2262 - val_ac
curacy: 0.8073 - val_loss: 0.6857
Epoch 16/50
313/313 ━━━━━━━━━━━━━━━━━━━━ 7s 22ms/step - accuracy: 0.9218 - loss: 0.2194 - val_ac
curacy: 0.8267 - val_loss: 0.5877
Epoch 17/50
313/313 ━━━━━━━━━━━━━━━━━━━━ 7s 22ms/step - accuracy: 0.9277 - loss: 0.2006 - val_ac
curacy: 0.8227 - val_loss: 0.6416
Epoch 18/50
313/313 ━━━━━━━━━━━━━━━━━━━━ 7s 22ms/step - accuracy: 0.9353 - loss: 0.1834 - val_ac
curacy: 0.8357 - val_loss: 0.6070
```

```
Epoch 19/50
313/313 ━━━━━━━━━━━━━━━━━━━━ 7s 22ms/step - accuracy: 0.9394 - loss: 0.1736 - val_ac
curacy: 0.8409 - val_loss: 0.5925
Epoch 20/50
313/313 ━━━━━━━━━━━━━━━━━━━━ 7s 22ms/step - accuracy: 0.9413 - loss: 0.1624 - val_ac
curacy: 0.8218 - val_loss: 0.6748
Epoch 21/50
313/313 ━━━━━━━━━━━━━━━━━━━━ 7s 22ms/step - accuracy: 0.9435 - loss: 0.1591 - val_ac
curacy: 0.8375 - val_loss: 0.6264
Epoch 22/50
313/313 ━━━━━━━━━━━━━━━━━━━━ 7s 22ms/step - accuracy: 0.9496 - loss: 0.1437 - val_ac
curacy: 0.8269 - val_loss: 0.6765
Epoch 23/50
313/313 ━━━━━━━━━━━━━━━━━━━━ 7s 22ms/step - accuracy: 0.9531 - loss: 0.1331 - val_ac
curacy: 0.8444 - val_loss: 0.6285
Epoch 24/50
313/313 ━━━━━━━━━━━━━━━━━━━━ 7s 22ms/step - accuracy: 0.9539 - loss: 0.1336 - val_ac
curacy: 0.8418 - val_loss: 0.6285
Epoch 25/50
313/313 ━━━━━━━━━━━━━━━━━━━━ 7s 22ms/step - accuracy: 0.9571 - loss: 0.1221 - val_ac
curacy: 0.8451 - val_loss: 0.6326
Epoch 26/50
313/313 ━━━━━━━━━━━━━━━━━━━━ 7s 22ms/step - accuracy: 0.9549 - loss: 0.1246 - val_ac
curacy: 0.8356 - val_loss: 0.6607
Epoch 27/50
313/313 ━━━━━━━━━━━━━━━━━━━━ 7s 22ms/step - accuracy: 0.9560 - loss: 0.1239 - val_ac
curacy: 0.8482 - val_loss: 0.6069
Epoch 28/50
313/313 ━━━━━━━━━━━━━━━━━━━━ 7s 22ms/step - accuracy: 0.9599 - loss: 0.1126 - val_ac
curacy: 0.8322 - val_loss: 0.7205
Epoch 29/50
313/313 ━━━━━━━━━━━━━━━━━━━━ 7s 22ms/step - accuracy: 0.9649 - loss: 0.0998 - val_ac
curacy: 0.8368 - val_loss: 0.6967
Epoch 30/50
313/313 ━━━━━━━━━━━━━━━━━━━━ 7s 22ms/step - accuracy: 0.9643 - loss: 0.1004 - val_ac
curacy: 0.8456 - val_loss: 0.6329
Epoch 31/50
313/313 ━━━━━━━━━━━━━━━━━━━━ 7s 22ms/step - accuracy: 0.9640 - loss: 0.1020 - val_ac
curacy: 0.8481 - val_loss: 0.6456
Epoch 32/50
313/313 ━━━━━━━━━━━━━━━━━━━━ 7s 22ms/step - accuracy: 0.9660 - loss: 0.0960 - val_ac
curacy: 0.8501 - val_loss: 0.6403
Epoch 33/50
313/313 ━━━━━━━━━━━━━━━━━━━━ 7s 22ms/step - accuracy: 0.9640 - loss: 0.1032 - val_ac
curacy: 0.8498 - val_loss: 0.6280
Epoch 34/50
313/313 ━━━━━━━━━━━━━━━━━━━━ 7s 22ms/step - accuracy: 0.9669 - loss: 0.0938 - val_ac
curacy: 0.8319 - val_loss: 0.7336
Epoch 35/50
313/313 ━━━━━━━━━━━━━━━━━━━━ 7s 22ms/step - accuracy: 0.9691 - loss: 0.0876 - val_ac
curacy: 0.8485 - val_loss: 0.6619
Epoch 36/50
313/313 ━━━━━━━━━━━━━━━━━━━━ 7s 22ms/step - accuracy: 0.9696 - loss: 0.0884 - val_ac
curacy: 0.8528 - val_loss: 0.6404
Epoch 37/50
313/313 ━━━━━━━━━━━━━━━━━━━━ 7s 22ms/step - accuracy: 0.9715 - loss: 0.0817 - val_ac
```

```
curacy: 0.8459 - val_loss: 0.6618
Epoch 38/50
313/313 ━━━━━━━━━━━━━━━━━━━━ 7s 23ms/step - accuracy: 0.9691 - loss: 0.0868 - val_ac
curacy: 0.8476 - val_loss: 0.6673
Epoch 39/50
313/313 ━━━━━━━━━━━━━━━━━━━━ 7s 22ms/step - accuracy: 0.9720 - loss: 0.0792 - val_ac
curacy: 0.8545 - val_loss: 0.6194
Epoch 40/50
313/313 ━━━━━━━━━━━━━━━━━━━━ 7s 23ms/step - accuracy: 0.9740 - loss: 0.0754 - val_ac
curacy: 0.8330 - val_loss: 0.7526
Epoch 41/50
313/313 ━━━━━━━━━━━━━━━━━━━━ 7s 22ms/step - accuracy: 0.9742 - loss: 0.0744 - val_ac
curacy: 0.8487 - val_loss: 0.6946
Epoch 42/50
313/313 ━━━━━━━━━━━━━━━━━━━━ 7s 23ms/step - accuracy: 0.9732 - loss: 0.0776 - val_ac
curacy: 0.8438 - val_loss: 0.7054
Epoch 43/50
313/313 ━━━━━━━━━━━━━━━━━━━━ 7s 22ms/step - accuracy: 0.9735 - loss: 0.0743 - val_ac
curacy: 0.8457 - val_loss: 0.6859
Epoch 44/50
313/313 ━━━━━━━━━━━━━━━━━━━━ 8s 25ms/step - accuracy: 0.9752 - loss: 0.0724 - val_ac
curacy: 0.8558 - val_loss: 0.6639
Epoch 45/50
313/313 ━━━━━━━━━━━━━━━━━━━━ 7s 22ms/step - accuracy: 0.9744 - loss: 0.0735 - val_ac
curacy: 0.8459 - val_loss: 0.6749
Epoch 46/50
313/313 ━━━━━━━━━━━━━━━━━━━━ 7s 23ms/step - accuracy: 0.9748 - loss: 0.0717 - val_ac
curacy: 0.8454 - val_loss: 0.7047
Epoch 47/50
313/313 ━━━━━━━━━━━━━━━━━━━━ 7s 22ms/step - accuracy: 0.9760 - loss: 0.0700 - val_ac
curacy: 0.8464 - val_loss: 0.6930
Epoch 48/50
313/313 ━━━━━━━━━━━━━━━━━━━━ 7s 22ms/step - accuracy: 0.9766 - loss: 0.0662 - val_ac
curacy: 0.8468 - val_loss: 0.7126
Epoch 49/50
313/313 ━━━━━━━━━━━━━━━━━━━━ 7s 22ms/step - accuracy: 0.9763 - loss: 0.0678 - val_ac
curacy: 0.8408 - val_loss: 0.7497
Epoch 50/50
313/313 ━━━━━━━━━━━━━━━━━━━━ 8s 25ms/step - accuracy: 0.9769 - loss: 0.0655 - val_ac
curacy: 0.8515 - val_loss: 0.7053
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.
saving.save_model(model)`. This file format is considered legacy. We recommend using
instead the native Keras format, e.g. `model.save('my_model.keras')` or `keras.savin
g.save_model(model, 'my_model.keras')`.
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.
mount("/content/drive", force_remount=True).
[INFO]: Evaluando el modelo...
79/79 ━━━━━━━━━━━━━━━━━━━━ 2s 17ms/step
              precision    recall  f1-score   support

       Avión       0.89      0.86      0.87      1000
   Automóvil       0.94      0.93      0.93      1000
      Pájaro       0.88      0.72      0.79      1000
        Gato       0.69      0.75      0.72      1000
      Ciervo       0.82      0.84      0.83      1000
       Perro       0.75      0.79      0.77      1000
        Rana       0.89      0.88      0.88      1000
     Caballo       0.85      0.90      0.88      1000
       Barco       0.94      0.90      0.92      1000
      Camión       0.89      0.92      0.91      1000

    accuracy                           0.85     10000
   macro avg       0.85      0.85      0.85     10000
weighted avg       0.85      0.85      0.85     10000
```



REDUCIENDO OVERFITTING MEDIANTE DATA AUGMENTATION

- Acondicionando dataset

In [17]:
```
from sklearn.preprocessing import LabelBinarizer
# Por si es necesario convertir a one-hot encoding
lb = LabelBinarizer()
trainY = lb.fit_transform(trainY)
testY = lb.transform(testY)
print(trainY.shape)
print(testY.shape)
```

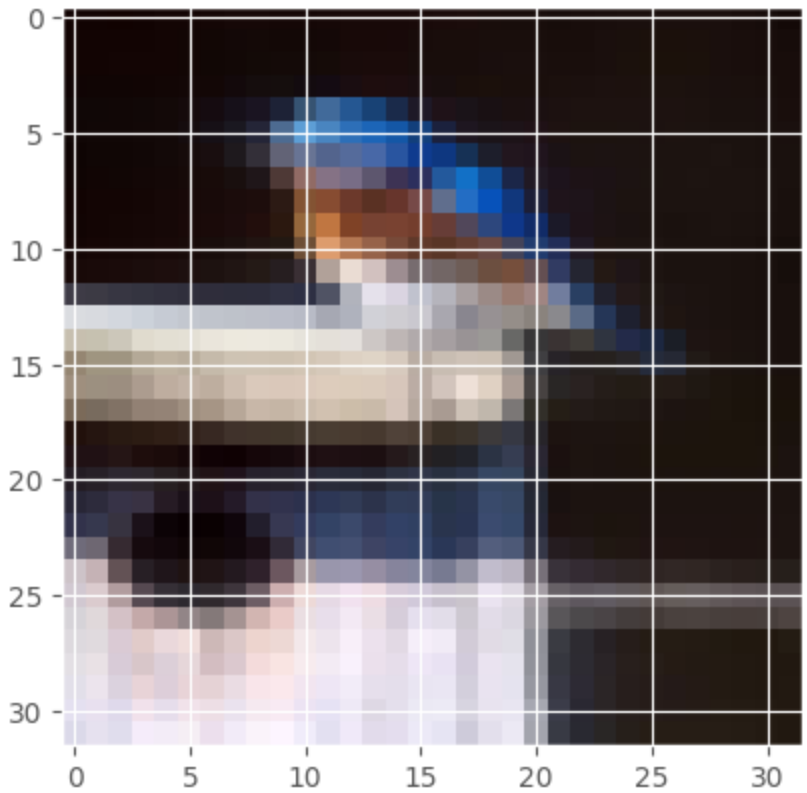
```
(50000, 10)
(10000, 10)
```

- Creando un contenedor DataGenerator para el aumento automático de muestras

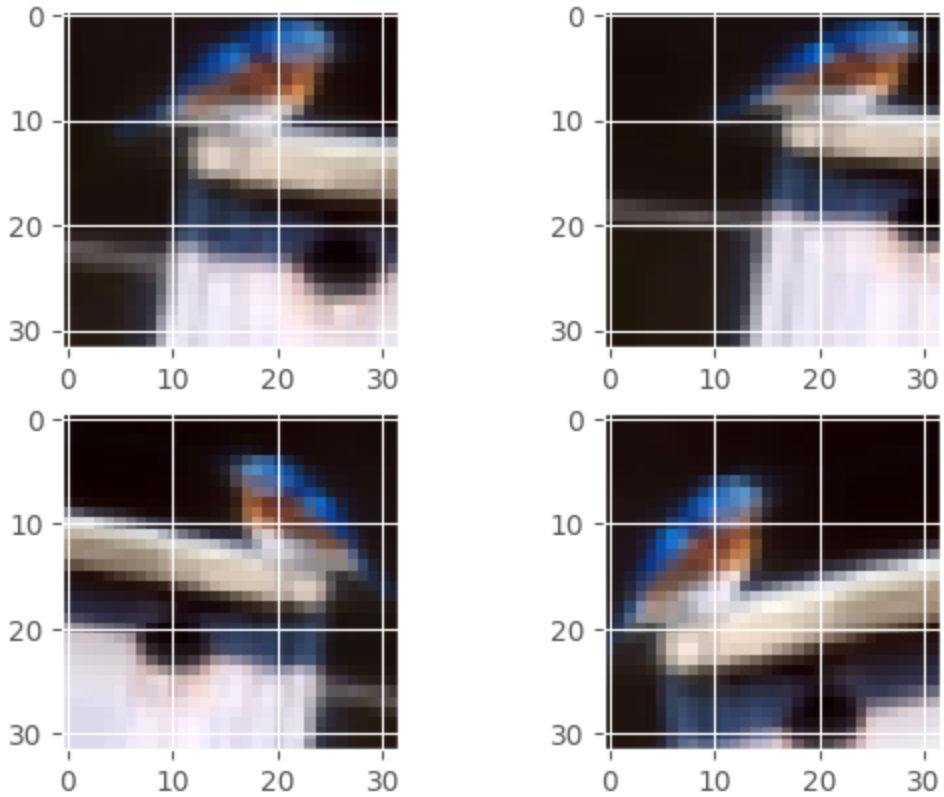
In [18]:
```
from tensorflow.keras.preprocessing.image import ImageDataGenerator

datagen = ImageDataGenerator(
    rotation_range=15, # grados de rotacion aleatoria
    width_shift_range=0.2, # fraccion del total (1) para mover la imagen
    height_shift_range=0.2, # fraccion del total (1) para mover la imagen
    horizontal_flip=True, # girar las imagenes horizontalmente (eje vertical)
    # shear_range=0, # deslizamiento
    zoom_range=0.2, # rango de zoom
    # fill_mode='nearest', # como rellenar posibles nuevos pixeles
    # channel_shift_range=0.2 # cambios aleatorios en los canales de la imagen
    validation_split=0.2,
)
```

- Inspeccionando las muestras generadas sintéticamente

In [19]:
```
from tensorflow.keras.preprocessing import image
import matplotlib.pyplot as plt
%matplotlib inline

sample = 13
plt.imshow(image.array_to_img(trainX[sample]))
plt.show()
print('Label = {}'.format(labelNames[trainY[sample].argmax(axis=0)]))

fig, axes = plt.subplots(2,2)
i = 0
for batch in datagen.flow(trainX[sample].reshape((1,32,32,3)),batch_size=1):
    #plt.figure(i)
    axes[i//2,i%2].imshow(image.array_to_img(batch[0]))
    i += 1
    if i == 4:
        break
plt.show()
```




```
Label = Pájaro
```



- Creando la topología de red neuronal y entrenándola: CNN

In [20]:
```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.
mount("/content/drive", force_remount=True).
```

In [21]:
```
!ls '/content/drive/MyDrive'
```

```
'2025-07-04 22-13-18.mkv'
'ahora realice el informe psicopedagogico que entr....gdoc'
 archivo
'Colab Notebooks'
'de esto cambia a modelo de tablas de 3fn con su p....gsheet'
'Examen Mejia Erick.pdf'
 S
```

In [22]:
```
# Import the necessary packages
import numpy as np
from tensorflow.keras import backend as K
from tensorflow.keras.layers import Input, Conv2D, Activation, Flatten, Dense, Drop
from tensorflow.keras.models import Model
from tensorflow.keras.models import Sequential
from tensorflow.keras.optimizers import SGD, Adam
from sklearn.metrics import classification_report
import matplotlib.pyplot as plt
from google.colab import drive

#########################################
###### Definimos la arquitectura ########
#########################################
# Definimos entradas
inp = Input(shape=(trainX.shape[1], trainX.shape[2], trainX.shape[3]))

# Primer set de capas CONV => RELU => CONV => RELU => POOL
x1 = Conv2D(32, (3, 3), padding="same", activation="relu")(inp)
x1 = BatchNormalization()(x1)
x1 = Conv2D(32, (3, 3), padding="same", activation="relu")(x1)
x1 = BatchNormalization()(x1)
x1 = MaxPooling2D(pool_size=(2, 2))(x1)
x1 = Dropout(0.25)(x1)

# Segundo set de capas CONV => RELU => CONV => RELU => POOL
x2 = Conv2D(64, (3, 3), padding="same", activation="relu")(x1)
x2 = BatchNormalization()(x2)
x2 = Conv2D(64, (3, 3), padding="same", activation="relu")(x2)
x2 = BatchNormalization()(x2)
x2 = MaxPooling2D(pool_size=(2, 2))(x2)
x2 = Dropout(0.25)(x2)

# Segundo set de capas CONV => RELU => CONV => RELU => POOL
x2 = Conv2D(256, (3, 3), padding="same", activation="relu")(x2)
x2 = BatchNormalization()(x2)
x2 = Conv2D(256, (3, 3), padding="same", activation="relu")(x2)
x2 = BatchNormalization()(x2)
x2 = MaxPooling2D(pool_size=(2, 2))(x2)
x2 = Dropout(0.25)(x2)

# Primer (y único) set de capas FC => RELU
xfc = Flatten()(x2)
```

```
xfc = Dense(512, activation="relu")(xfc)
xfc = BatchNormalization()(xfc)
xfc = Dropout(0.5)(xfc)
# Clasificador softmax
predictions = Dense(10, activation="softmax")(xfc)

# Unimos las entradas y el modelo mediante la función Model con parámetros inputs y
model_aug = Model(inputs=inp, outputs=predictions)

# Compilar el modelo
print("[INFO]: Compilando el modelo...")
model_aug.compile(loss="categorical_crossentropy", optimizer=Adam(learning_rate=0.0

# Entrenamiento de la red
print("[INFO]: Entrenando la red...")
H_aug = model_aug.fit(datagen.flow(trainX, trainY, batch_size=128, subset="training
                                   steps_per_epoch = len(trainX)// 128, epochs=50, val
# Almaceno el modelo en Drive
# Montamos la unidad de Drive
drive.mount('/content/drive')
# Almacenamos el modelo empleando la función mdoel.save de Keras
model_aug.save(BASE_FOLDER+"deepCNN_CIFAR10_aug.h5")

# Evaluación del modelo
print("[INFO]: Evaluando el modelo...")
# Efectuamos la predicción (empleamos el mismo valor de batch_size que en training)
predictions = model_aug.predict(testX, batch_size=128)
# Sacamos el report para test
print(classification_report(testY.argmax(axis=1), predictions.argmax(axis=1), targe

# Gráficas
plt.style.use("ggplot")
plt.figure()
plt.plot(np.arange(0, 50), H_aug.history["loss"], label="train_loss")
plt.plot(np.arange(0, 50), H_aug.history["val_loss"], label="val_loss")
plt.plot(np.arange(0, 50), H_aug.history["accuracy"], label="train_acc")
plt.plot(np.arange(0, 50), H_aug.history["val_accuracy"], label="val_acc")
plt.title("Training Loss and Accuracy")
plt.xlabel("Epoch #")
plt.ylabel("Loss/Accuracy")
plt.legend()
plt.show()
```

```
[INFO]: Compilando el modelo...
[INFO]: Entrenando la red...
Epoch 1/50
313/390 ━━━━━━━━━━━━━━━━━━━━ 8s 113ms/step - accuracy: 0.3042 - loss: 2.2853
/usr/local/lib/python3.12/dist-packages/keras/src/trainers/epoch_iterator.py:116: Us
erWarning: Your input ran out of data; interrupting training. Make sure that your da
taset or generator can generate at least `steps_per_epoch * epochs` batches. You may
need to use the `.repeat()` function when building your dataset.
  self._interrupted_warning()
```

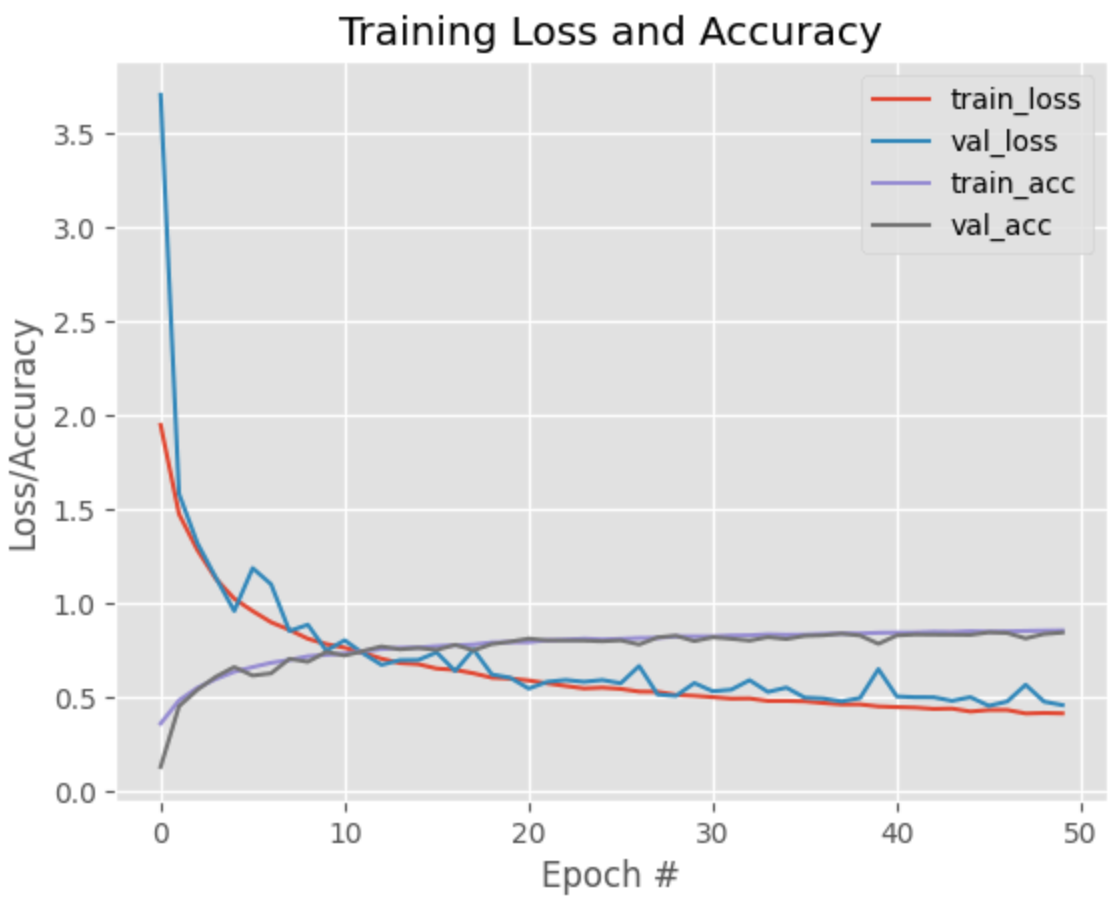
```
□□□
```

```
ccuracy: 0.7835 - val_loss: 0.6214
Epoch 20/50
390/390 ━━━━━━━━━━━━━━━━━━━━ 39s 100ms/step - accuracy: 0.7936 - loss: 0.5998 - val_
accuracy: 0.7961 - val_loss: 0.6040
Epoch 21/50
390/390 ━━━━━━━━━━━━━━━━━━━━ 39s 100ms/step - accuracy: 0.7934 - loss: 0.5897 - val_
accuracy: 0.8124 - val_loss: 0.5465
Epoch 22/50
390/390 ━━━━━━━━━━━━━━━━━━━━ 38s 97ms/step - accuracy: 0.8036 - loss: 0.5743 - val_a
ccuracy: 0.8031 - val_loss: 0.5823
Epoch 23/50
390/390 ━━━━━━━━━━━━━━━━━━━━ 39s 99ms/step - accuracy: 0.8064 - loss: 0.5607 - val_a
ccuracy: 0.8028 - val_loss: 0.5922
Epoch 24/50
390/390 ━━━━━━━━━━━━━━━━━━━━ 49s 126ms/step - accuracy: 0.8122 - loss: 0.5474 - val_
accuracy: 0.8020 - val_loss: 0.5825
Epoch 25/50
390/390 ━━━━━━━━━━━━━━━━━━━━ 39s 101ms/step - accuracy: 0.8082 - loss: 0.5515 - val_
accuracy: 0.7985 - val_loss: 0.5921
Epoch 26/50
390/390 ━━━━━━━━━━━━━━━━━━━━ 39s 101ms/step - accuracy: 0.8113 - loss: 0.5449 - val_
accuracy: 0.8032 - val_loss: 0.5748
Epoch 27/50
390/390 ━━━━━━━━━━━━━━━━━━━━ 38s 97ms/step - accuracy: 0.8155 - loss: 0.5304 - val_a
ccuracy: 0.7809 - val_loss: 0.6668
Epoch 28/50
390/390 ━━━━━━━━━━━━━━━━━━━━ 39s 100ms/step - accuracy: 0.8164 - loss: 0.5292 - val_
accuracy: 0.8188 - val_loss: 0.5154
Epoch 29/50
390/390 ━━━━━━━━━━━━━━━━━━━━ 39s 101ms/step - accuracy: 0.8242 - loss: 0.5128 - val_
accuracy: 0.8276 - val_loss: 0.5076
Epoch 30/50
390/390 ━━━━━━━━━━━━━━━━━━━━ 38s 97ms/step - accuracy: 0.8245 - loss: 0.5081 - val_a
ccuracy: 0.7993 - val_loss: 0.5759
Epoch 31/50
390/390 ━━━━━━━━━━━━━━━━━━━━ 39s 101ms/step - accuracy: 0.8243 - loss: 0.5010 - val_
accuracy: 0.8199 - val_loss: 0.5322
Epoch 32/50
390/390 ━━━━━━━━━━━━━━━━━━━━ 40s 103ms/step - accuracy: 0.8287 - loss: 0.4931 - val_
accuracy: 0.8119 - val_loss: 0.5406
Epoch 33/50
390/390 ━━━━━━━━━━━━━━━━━━━━ 38s 98ms/step - accuracy: 0.8300 - loss: 0.4938 - val_a
ccuracy: 0.8018 - val_loss: 0.5911
Epoch 34/50
390/390 ━━━━━━━━━━━━━━━━━━━━ 40s 101ms/step - accuracy: 0.8352 - loss: 0.4807 - val_
accuracy: 0.8206 - val_loss: 0.5286
Epoch 35/50
390/390 ━━━━━━━━━━━━━━━━━━━━ 40s 104ms/step - accuracy: 0.8328 - loss: 0.4808 - val_
accuracy: 0.8103 - val_loss: 0.5517
Epoch 36/50
390/390 ━━━━━━━━━━━━━━━━━━━━ 39s 98ms/step - accuracy: 0.8339 - loss: 0.4786 - val_a
ccuracy: 0.8272 - val_loss: 0.4983
Epoch 37/50
390/390 ━━━━━━━━━━━━━━━━━━━━ 41s 98ms/step - accuracy: 0.8376 - loss: 0.4698 - val_a
ccuracy: 0.8301 - val_loss: 0.4937
Epoch 38/50
```

```
390/390 ━━━━━━━━━━━━━━━━━━━━ 40s 103ms/step - accuracy: 0.8396 - loss: 0.4624 - val_
accuracy: 0.8389 - val_loss: 0.4779
Epoch 39/50
390/390 ━━━━━━━━━━━━━━━━━━━━ 40s 102ms/step - accuracy: 0.8397 - loss: 0.4626 - val_
accuracy: 0.8291 - val_loss: 0.4959
Epoch 40/50
390/390 ━━━━━━━━━━━━━━━━━━━━ 40s 100ms/step - accuracy: 0.8435 - loss: 0.4514 - val_
accuracy: 0.7848 - val_loss: 0.6502
Epoch 41/50
390/390 ━━━━━━━━━━━━━━━━━━━━ 50s 128ms/step - accuracy: 0.8445 - loss: 0.4480 - val_
accuracy: 0.8295 - val_loss: 0.5046
Epoch 42/50
390/390 ━━━━━━━━━━━━━━━━━━━━ 40s 103ms/step - accuracy: 0.8448 - loss: 0.4454 - val_
accuracy: 0.8346 - val_loss: 0.5007
Epoch 43/50
390/390 ━━━━━━━━━━━━━━━━━━━━ 40s 102ms/step - accuracy: 0.8487 - loss: 0.4384 - val_
accuracy: 0.8338 - val_loss: 0.5003
Epoch 44/50
390/390 ━━━━━━━━━━━━━━━━━━━━ 39s 99ms/step - accuracy: 0.8476 - loss: 0.4402 - val_a
ccuracy: 0.8333 - val_loss: 0.4814
Epoch 45/50
390/390 ━━━━━━━━━━━━━━━━━━━━ 40s 103ms/step - accuracy: 0.8519 - loss: 0.4243 - val_
accuracy: 0.8325 - val_loss: 0.5008
Epoch 46/50
390/390 ━━━━━━━━━━━━━━━━━━━━ 39s 99ms/step - accuracy: 0.8496 - loss: 0.4325 - val_a
ccuracy: 0.8457 - val_loss: 0.4545
Epoch 47/50
390/390 ━━━━━━━━━━━━━━━━━━━━ 39s 99ms/step - accuracy: 0.8512 - loss: 0.4326 - val_a
ccuracy: 0.8425 - val_loss: 0.4777
Epoch 48/50
390/390 ━━━━━━━━━━━━━━━━━━━━ 40s 102ms/step - accuracy: 0.8534 - loss: 0.4139 - val_
accuracy: 0.8134 - val_loss: 0.5668
Epoch 49/50
390/390 ━━━━━━━━━━━━━━━━━━━━ 39s 100ms/step - accuracy: 0.8550 - loss: 0.4165 - val_
accuracy: 0.8372 - val_loss: 0.4771
Epoch 50/50
390/390 ━━━━━━━━━━━━━━━━━━━━ 40s 98ms/step - accuracy: 0.8561 - loss: 0.4146 - val_a
ccuracy: 0.8446 - val_loss: 0.4585
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.
saving.save_model(model)`. This file format is considered legacy. We recommend using
instead the native Keras format, e.g. `model.save('my_model.keras')` or `keras.savin
g.save_model(model, 'my_model.keras')`.
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.
mount("/content/drive", force_remount=True).
[INFO]: Evaluando el modelo...
79/79 ━━━━━━━━━━━━━━━━━━━━ 2s 17ms/step
              precision    recall  f1-score   support

       Avión       0.94      0.82      0.88      1000
   Automóvil       0.95      0.93      0.94      1000
      Pájaro       0.85      0.83      0.84      1000
        Gato       0.80      0.72      0.76      1000
      Ciervo       0.79      0.92      0.85      1000
       Perro       0.87      0.77      0.81      1000
        Rana       0.82      0.95      0.88      1000
     Caballo       0.92      0.88      0.90      1000
       Barco       0.95      0.92      0.93      1000
      Camión       0.86      0.96      0.91      1000

    accuracy                           0.87     10000
   macro avg       0.87      0.87      0.87     10000
weighted avg       0.87      0.87      0.87     10000
```



EJERCICIO PROPUESTO: Modificar la ejecución con Data Augmentation para que sea directamente comparable a la ejecución por defecto. Para ello, el validation data debe ser correcto. Pista: Llevar a cabo el validation split cuando se instancia el objeto ImageDataGenerator. El método flow también se verá afectado (leer documentación).

TRABAJANDO CON REDES PRE-ENTRENADAS: TRANSFER LEARNING & FINE-TUNING

- Cargando el conjunto de datos y acondicionándolo como en la VGG

In [1]:
```
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.applications import imagenet_utils
from sklearn.preprocessing import LabelBinarizer

# Importando y normalizando el set de datos CIFAR10
print("[INFO]: Loading CIFAR-10 data...")
((trainX, trainY), (testX, testY)) = cifar10.load_data()
labelNames = ["Avión", "Automóvil", "Pájaro", "Gato", "Ciervo", "Perro", "Rana", "C

#One-hot encoding
lb = LabelBinarizer()
trainY = lb.fit_transform(trainY)
testY = lb.transform(testY)

# IMPORTANTE: Se normalizan los datos como se normalizaron en el entrenamiento con
trainX = imagenet_utils.preprocess_input(trainX)
testX = imagenet_utils.preprocess_input(testX)

print(trainX.shape)
#print(trainY.shape)
```

```
[INFO]: Loading CIFAR-10 data...
Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
170498071/170498071 ━━━━━━━━━━━━━━━━━━━━ 3s 0us/step
(50000, 32, 32, 3)
```

- Cargando la topología de CNN (base model)

In [2]:
```
#keras incluye varias arquitecturas
# VGG16, VGG19, ResNet50, Xception, InceptionV3, InceptionResNetV2, MobileNetV2, De
# documentacion https://keras.io/applications/
# Visual Geometry Group 16 / 19 (numero de layers)
# 1 y 2 en la competicion ImageNet 2014
# Kernels pequeños de 3x3

from tensorflow.keras.applications import VGG16

base_model = VGG16(weights='imagenet',
                   include_top=False, # No incluir el top model, i.e. la parte densa
                   input_shape=(32,32,3))
base_model.summary()
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/v
gg16/vgg16_weights_tf_dim_ordering_tf_kernels_notop.h5
58889256/58889256 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Model: "vgg16"
```

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 32, 32, 3)	0
block1_conv1 (Conv2D)	(None, 32, 32, 64)	1,792
block1_conv2 (Conv2D)	(None, 32, 32, 64)	36,928
block1_pool (MaxPooling2D)	(None, 16, 16, 64)	0
block2_conv1 (Conv2D)	(None, 16, 16, 128)	73,856
block2_conv2 (Conv2D)	(None, 16, 16, 128)	147,584
block2_pool (MaxPooling2D)	(None, 8, 8, 128)	0
block3_conv1 (Conv2D)	(None, 8, 8, 256)	295,168
block3_conv2 (Conv2D)	(None, 8, 8, 256)	590,080
block3_conv3 (Conv2D)	(None, 8, 8, 256)	590,080
block3_pool (MaxPooling2D)	(None, 4, 4, 256)	0
block4_conv1 (Conv2D)	(None, 4, 4, 512)	1,180,160
block4_conv2 (Conv2D)	(None, 4, 4, 512)	2,359,808
block4_conv3 (Conv2D)	(None, 4, 4, 512)	2,359,808
block4_pool (MaxPooling2D)	(None, 2, 2, 512)	0
block5_conv1 (Conv2D)	(None, 2, 2, 512)	2,359,808
block5_conv2 (Conv2D)	(None, 2, 2, 512)	2,359,808
block5_conv3 (Conv2D)	(None, 2, 2, 512)	2,359,808
block5_pool (MaxPooling2D)	(None, 1, 1, 512)	0

Total params: 14,714,688 (56.13 MB)

Trainable params: 14,714,688 (56.13 MB)

Non-trainable params: 0 (0.00 B)

- Creando el top model y congelando TODAS las capas convolucionales (TRANSFER LEARNING)

In [3]:
```
# conectarlo a nueva parte densa
from tensorflow.keras.models import Sequential
from tensorflow.keras import layers

base_model.trainable = False # Evitar que los pesos se modifiquen en la parte convo
pre_trained_model = Sequential()
```

```
pre_trained_model.add(base_model)
pre_trained_model.add(layers.Flatten())
pre_trained_model.add(layers.Dense(256, activation='relu', name="densa_1"))
pre_trained_model.add(layers.Dense(10, activation='softmax'))

pre_trained_model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
vgg16 (Functional)	(None, 1, 1, 512)	14,714,688
flatten (Flatten)	(None, 512)	0
densa_1 (Dense)	(None, 256)	131,328
dense (Dense)	(None, 10)	2,570

Total params: 14,848,586 (56.64 MB)

Trainable params: 133,898 (523.04 KB)

Non-trainable params: 14,714,688 (56.13 MB)

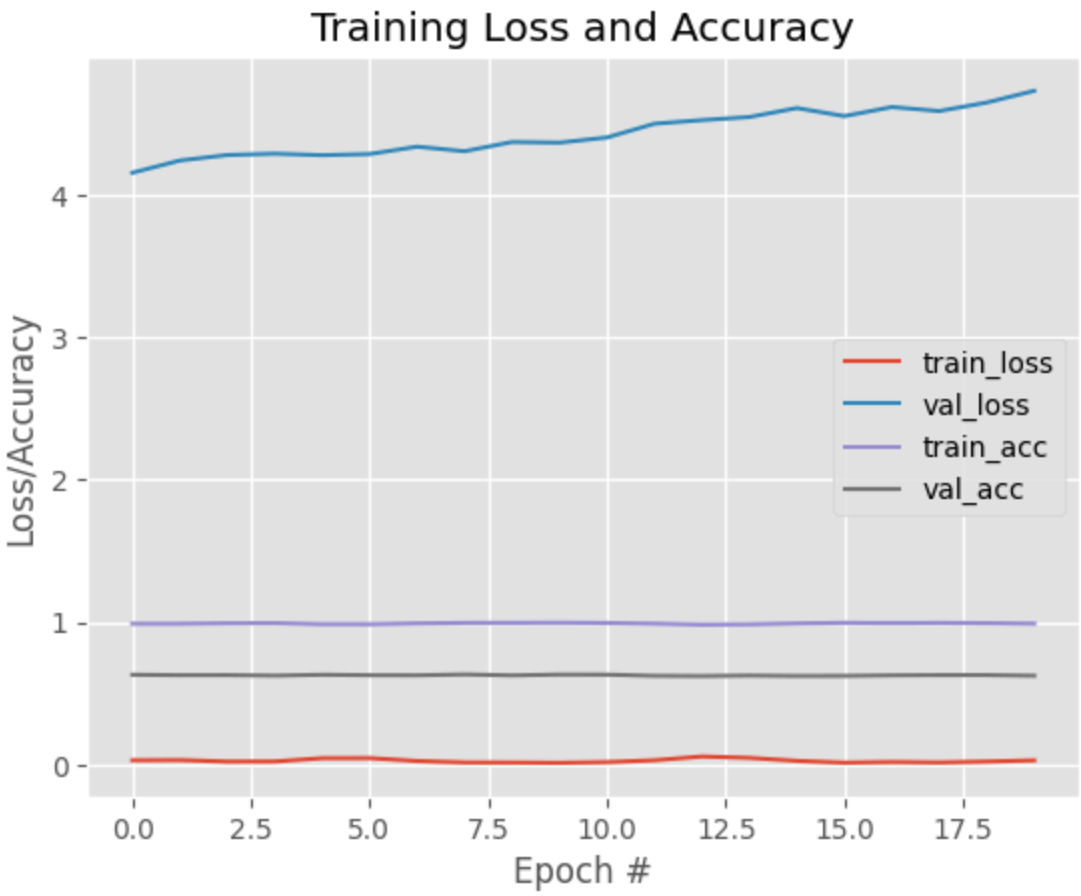
- Entrenando la solución

In [7]:
```
import numpy as np
from tensorflow.keras import backend as K
from tensorflow.keras.layers import Input, Conv2D, Activation, Flatten, Dense, Drop
from tensorflow.keras.models import Model
from tensorflow.keras.models import Sequential
from tensorflow.keras.optimizers import SGD, Adam
from sklearn.metrics import classification_report
import matplotlib.pyplot as plt
from google.colab import drive

BASE_FOLDER = '/content/drive/My Drive/archivo/'

# Compilar el modelo
print("[INFO]: Compilando el modelo...")
pre_trained_model.compile(loss="categorical_crossentropy", optimizer=Adam(learning_

# Entrenamiento de la red
print("[INFO]: Entrenando la red...")
H_pre = pre_trained_model.fit(trainX, trainY, batch_size=128, epochs=20, validation
# Almaceno el modelo en Drive
# Montamos la unidad de Drive
drive.mount('/content/drive')
# Almacenamos el modelo empleando la función mdoel.save de Keras
pre_trained_model.save(BASE_FOLDER+"deepCNN_CIFAR10_pretrained.h5") #(X)

# Evaluación del modelo
print("[INFO]: Evaluando el modelo...")
# Efectuamos la predicción (empleamos el mismo valor de batch_size que en training)
predictions = pre_trained_model.predict(testX, batch_size=128)
```

```python
# Sacamos el report para test
print(classification_report(testY.argmax(axis=1), predictions.argmax(axis=1), targe

# Gráficas
plt.style.use("ggplot")
plt.figure()
plt.plot(np.arange(0, 20), H_pre.history["loss"], label="train_loss")
plt.plot(np.arange(0, 20), H_pre.history["val_loss"], label="val_loss")
plt.plot(np.arange(0, 20), H_pre.history["accuracy"], label="train_acc")
plt.plot(np.arange(0, 20), H_pre.history["val_accuracy"], label="val_acc")
plt.title("Training Loss and Accuracy")
plt.xlabel("Epoch #")
plt.ylabel("Loss/Accuracy")
plt.legend()
plt.show()
```

```
[INFO]: Compilando el modelo...
[INFO]: Entrenando la red...
Epoch 1/20
313/313 ━━━━━━━━━━━━━━━━━━━━ 14s 39ms/step - accuracy: 0.9893 - loss: 0.0305 - val_a
ccuracy: 0.6312 - val_loss: 4.1573
Epoch 2/20
313/313 ━━━━━━━━━━━━━━━━━━━━ 10s 31ms/step - accuracy: 0.9894 - loss: 0.0319 - val_a
ccuracy: 0.6282 - val_loss: 4.2431
Epoch 3/20
313/313 ━━━━━━━━━━━━━━━━━━━━ 10s 32ms/step - accuracy: 0.9931 - loss: 0.0214 - val_a
ccuracy: 0.6287 - val_loss: 4.2817
Epoch 4/20
313/313 ━━━━━━━━━━━━━━━━━━━━ 10s 30ms/step - accuracy: 0.9941 - loss: 0.0219 - val_a
ccuracy: 0.6250 - val_loss: 4.2910
Epoch 5/20
313/313 ━━━━━━━━━━━━━━━━━━━━ 9s 29ms/step - accuracy: 0.9858 - loss: 0.0451 - val_ac
curacy: 0.6311 - val_loss: 4.2806
Epoch 6/20
313/313 ━━━━━━━━━━━━━━━━━━━━ 9s 28ms/step - accuracy: 0.9856 - loss: 0.0456 - val_ac
curacy: 0.6280 - val_loss: 4.2886
Epoch 7/20
313/313 ━━━━━━━━━━━━━━━━━━━━ 9s 27ms/step - accuracy: 0.9923 - loss: 0.0255 - val_ac
curacy: 0.6273 - val_loss: 4.3404
Epoch 8/20
313/313 ━━━━━━━━━━━━━━━━━━━━ 9s 28ms/step - accuracy: 0.9956 - loss: 0.0151 - val_ac
curacy: 0.6332 - val_loss: 4.3083
Epoch 9/20
313/313 ━━━━━━━━━━━━━━━━━━━━ 9s 28ms/step - accuracy: 0.9960 - loss: 0.0146 - val_ac
curacy: 0.6268 - val_loss: 4.3732
Epoch 10/20
313/313 ━━━━━━━━━━━━━━━━━━━━ 9s 30ms/step - accuracy: 0.9968 - loss: 0.0124 - val_ac
curacy: 0.6323 - val_loss: 4.3687
Epoch 11/20
313/313 ━━━━━━━━━━━━━━━━━━━━ 10s 31ms/step - accuracy: 0.9951 - loss: 0.0176 - val_a
ccuracy: 0.6318 - val_loss: 4.4050
Epoch 12/20
313/313 ━━━━━━━━━━━━━━━━━━━━ 10s 31ms/step - accuracy: 0.9902 - loss: 0.0306 - val_a
ccuracy: 0.6237 - val_loss: 4.5017
Epoch 13/20
313/313 ━━━━━━━━━━━━━━━━━━━━ 9s 29ms/step - accuracy: 0.9825 - loss: 0.0570 - val_ac
curacy: 0.6219 - val_loss: 4.5268
Epoch 14/20
313/313 ━━━━━━━━━━━━━━━━━━━━ 9s 29ms/step - accuracy: 0.9849 - loss: 0.0473 - val_ac
curacy: 0.6256 - val_loss: 4.5487
Epoch 15/20
313/313 ━━━━━━━━━━━━━━━━━━━━ 9s 28ms/step - accuracy: 0.9921 - loss: 0.0262 - val_ac
curacy: 0.6225 - val_loss: 4.6115
Epoch 16/20
313/313 ━━━━━━━━━━━━━━━━━━━━ 9s 28ms/step - accuracy: 0.9960 - loss: 0.0135 - val_ac
curacy: 0.6232 - val_loss: 4.5551
Epoch 17/20
313/313 ━━━━━━━━━━━━━━━━━━━━ 9s 29ms/step - accuracy: 0.9947 - loss: 0.0184 - val_ac
curacy: 0.6272 - val_loss: 4.6187
Epoch 18/20
313/313 ━━━━━━━━━━━━━━━━━━━━ 9s 29ms/step - accuracy: 0.9957 - loss: 0.0149 - val_ac
curacy: 0.6288 - val_loss: 4.5907
```

```
Epoch 19/20
313/313 ━━━━━━━━━━━━━━━━━━━━ 9s 29ms/step - accuracy: 0.9941 - loss: 0.0216 - val_ac
curacy: 0.6289 - val_loss: 4.6505
Epoch 20/20
313/313 ━━━━━━━━━━━━━━━━━━━━ 9s 30ms/step - accuracy: 0.9901 - loss: 0.0298 - val_ac
curacy: 0.6242 - val_loss: 4.7326
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.
saving.save_model(model)`. This file format is considered legacy. We recommend using
instead the native Keras format, e.g. `model.save('my_model.keras')` or `keras.savin
g.save_model(model, 'my_model.keras')`.
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.
mount("/content/drive", force_remount=True).
[INFO]: Evaluando el modelo...
79/79 ━━━━━━━━━━━━━━━━━━━━ 3s 31ms/step
              precision    recall  f1-score   support

       Avión       0.72      0.69      0.70      1000
   Automóvil       0.69      0.73      0.71      1000
      Pájaro       0.55      0.55      0.55      1000
        Gato       0.47      0.47      0.47      1000
      Ciervo       0.58      0.53      0.56      1000
       Perro       0.56      0.56      0.56      1000
        Rana       0.67      0.70      0.69      1000
     Caballo       0.68      0.67      0.67      1000
       Barco       0.74      0.75      0.75      1000
      Camión       0.67      0.69      0.68      1000

    accuracy                           0.63     10000
   macro avg       0.63      0.63      0.63     10000
weighted avg       0.63      0.63      0.63     10000
```



- Creando el top model y descongelando bloques convolucionales (FINE TUNING)

In [16]:
```
# Imports que vamos a necesitar
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.optimizers import SGD, Adam
from tensorflow.keras.applications import VGG16, imagenet_utils
from tensorflow.keras.utils import to_categorical
from tensorflow.keras import optimizers
from tensorflow.keras.layers import Dropout, Flatten, GlobalMaxPooling2D, Dense
from tensorflow.keras import Model
import matplotlib.pyplot as plt
from sklearn.metrics import classification_report
import numpy as np
from sklearn.preprocessing import LabelBinarizer

#Cargamos el dataset CIFAR10
(trainX, trainY), (testX, testY) = cifar10.load_data()

# Normalizamos las entradas de idéntica forma a como lo hicieron para entrenar la V
trainX = imagenet_utils.preprocess_input(trainX)
testX = imagenet_utils.preprocess_input(testX)

# Definimos dimensiones de nuestros datos de entrada y lista con las categorias de
input_shape = (32, 32, 3)
labelNames = ["Avión", "Automóvil", "Pájaro", "Gato", "Ciervo", "Perro", "Rana", "C
```

```python
# En caso de inestabilidades numéricas pasar datos a one-hot encoding
lb = LabelBinarizer()
trainY = lb.fit_transform(trainY)
testY = lb.transform(testY)

# Importamos VGG16 con pesos de imagenet y sin top_model especificando tamaño de en
base_model = VGG16(weights='imagenet', include_top=False, input_shape=input_shape)
# Mostramos la arquitectura
base_model.summary()

# Congelamos las capas de los 4 primeros bloques convolucionales, el quinto se re-e
# En base_model.layers.name tenemos la información del nombre de la capa
for layer in base_model.layers:
  if layer.name == 'block5_conv1':
    break
  layer.trainable = False
  print('Capa ' + layer.name + ' congelada...')

# Cogemos la última capa del model y le añadimos nuestro clasificador (top_model)
last = base_model.layers[-1].output #(X)
x = Flatten()(last) #(X)
x = Dense(1000, activation='relu', name='fc1')(x) #(X)
x= Dropout(0.3)(x) #(X)
x = Dense(200, activation='relu', name='fc2')(x) #(X)
x = Dense(10, activation='softmax', name='predictions')(x) #(X)
model = Model(base_model.input, x) #(X)

# Compilamos el modelo
optimizer=Adam(lr=0.1)
model.compile(optimizer=optimizer, loss='categorical_crossentropy', metrics=['accur

# Vamos a visualizar el modelo prestando especial atención en el número de pesos to
# ¿tiene sentido en comparación al ejemplo de transfer learning?
model.summary()

# Entrenamos el modelo
H = model.fit(trainX, trainY, validation_split=0.2, batch_size=64, epochs=20, verbo

# Evaluación del modelo
print("[INFO]: Evaluando el modelo...")
predictions = model.predict(testX, batch_size=64)
# Obtener el report de clasificación
print(classification_report(testY.argmax(axis=1), predictions.argmax(axis=1), targe

# Gráficas
plt.style.use("ggplot")
plt.figure()
plt.plot(np.arange(0, 20), H.history["loss"], label="train_loss")
plt.plot(np.arange(0, 20), H.history["val_loss"], label="val_loss")
plt.plot(np.arange(0, 20), H.history["accuracy"], label="train_acc")
plt.plot(np.arange(0, 20), H.history["val_accuracy"], label="val_acc")
plt.title("Training Loss and Accuracy")
plt.xlabel("Epoch #")
plt.ylabel("Loss/Accuracy")
plt.legend()
plt.show()
```

Model: "vgg16"

Layer (type)	Output Shape	Param #
input_layer_9 (InputLayer)	(None, 32, 32, 3)	0
block1_conv1 (Conv2D)	(None, 32, 32, 64)	1,792
block1_conv2 (Conv2D)	(None, 32, 32, 64)	36,928
block1_pool (MaxPooling2D)	(None, 16, 16, 64)	0
block2_conv1 (Conv2D)	(None, 16, 16, 128)	73,856
block2_conv2 (Conv2D)	(None, 16, 16, 128)	147,584
block2_pool (MaxPooling2D)	(None, 8, 8, 128)	0
block3_conv1 (Conv2D)	(None, 8, 8, 256)	295,168
block3_conv2 (Conv2D)	(None, 8, 8, 256)	590,080
block3_conv3 (Conv2D)	(None, 8, 8, 256)	590,080
block3_pool (MaxPooling2D)	(None, 4, 4, 256)	0
block4_conv1 (Conv2D)	(None, 4, 4, 512)	1,180,160
block4_conv2 (Conv2D)	(None, 4, 4, 512)	2,359,808
block4_conv3 (Conv2D)	(None, 4, 4, 512)	2,359,808
block4_pool (MaxPooling2D)	(None, 2, 2, 512)	0
block5_conv1 (Conv2D)	(None, 2, 2, 512)	2,359,808
block5_conv2 (Conv2D)	(None, 2, 2, 512)	2,359,808
block5_conv3 (Conv2D)	(None, 2, 2, 512)	2,359,808
block5_pool (MaxPooling2D)	(None, 1, 1, 512)	0

Total params: 14,714,688 (56.13 MB)

Trainable params: 14,714,688 (56.13 MB)

Non-trainable params: 0 (0.00 B)

```
Capa input_layer_9 congelada...
Capa block1_conv1 congelada...
Capa block1_conv2 congelada...
Capa block1_pool congelada...
Capa block2_conv1 congelada...
Capa block2_conv2 congelada...
Capa block2_pool congelada...
Capa block3_conv1 congelada...
Capa block3_conv2 congelada...
Capa block3_conv3 congelada...
Capa block3_pool congelada...
Capa block4_conv1 congelada...
Capa block4_conv2 congelada...
Capa block4_conv3 congelada...
Capa block4_pool congelada...
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
/tmp/ipykernel_4256/3812268626.py in <cell line: 0>()
     51
     52 # Compilamos el modelo
---> 53 optimizer=Adam(lr=0.1)
     54 model.compile(optimizer=optimizer, loss='categorical_crossentropy', metrics=
['accuracy'])
     55

/usr/local/lib/python3.12/dist-packages/keras/src/optimizers/adam.py in __init__(sel
f, learning_rate, beta_1, beta_2, epsilon, amsgrad, weight_decay, clipnorm, clipvalu
e, global_clipnorm, use_ema, ema_momentum, ema_overwrite_frequency, loss_scale_facto
r, gradient_accumulation_steps, name, **kwargs)
     60         **kwargs,
     61     ):
---> 62         super().__init__(
     63             learning_rate=learning_rate,
     64             name=name,

/usr/local/lib/python3.12/dist-packages/keras/src/backend/tensorflow/optimizer.py in
__init__(self, *args, **kwargs)
     19 class TFOptimizer(KerasAutoTrackable, base_optimizer.BaseOptimizer):
     20     def __init__(self, *args, **kwargs):
---> 21         super().__init__(*args, **kwargs)
     22         self._distribution_strategy = tf.distribute.get_strategy()
     23

/usr/local/lib/python3.12/dist-packages/keras/src/optimizers/base_optimizer.py in __
init__(self, learning_rate, weight_decay, clipnorm, clipvalue, global_clipnorm, use_
ema, ema_momentum, ema_overwrite_frequency, loss_scale_factor, gradient_accumulation
_steps, name, **kwargs)
     88             )
     89         if kwargs:
---> 90             raise ValueError(f"Argument(s) not recognized: {kwargs}")
     91
     92         if name is None:

ValueError: Argument(s) not recognized: {'lr': 0.1}
```

EJERCICIO PROPUESTO 1: Sacar el entrenamiento del mínimo local.

In [13]:

```python
# Imports que vamos a necesitar
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.optimizers import SGD, Adam
from tensorflow.keras.applications import VGG16, imagenet_utils
from tensorflow.keras.utils import to_categorical
from tensorflow.keras import optimizers
from tensorflow.keras.layers import Dropout, Flatten, GlobalMaxPooling2D, Dense
from tensorflow.keras import Model
import matplotlib.pyplot as plt
from sklearn.metrics import classification_report
import numpy as np
from sklearn.preprocessing import LabelBinarizer

#Cargamos el dataset CIFAR10
(trainX, trainY), (testX, testY) = cifar10.load_data()

# Normalizamos las entradas de idéntica forma a como lo hicieron para entrenar la V
trainX = imagenet_utils.preprocess_input(trainX)
testX = imagenet_utils.preprocess_input(testX)

# Definimos dimensiones de nuestros datos de entrada y lista con las categorias de
input_shape = (32, 32, 3)
labelNames = ["Avión", "Automóvil", "Pájaro", "Gato", "Ciervo", "Perro", "Rana", "C

# En caso de inestabilidades numéricas pasar datos a one-hot encoding
lb = LabelBinarizer()
trainY = lb.fit_transform(trainY)
testY = lb.transform(testY)

# Importamos VGG16 con pesos de imagenet y sin top_model especificando tamaño de en
base_model = VGG16(weights='imagenet', include_top=False, input_shape=input_shape)
# Mostramos la arquitectura
base_model.summary()

# Congelamos las capas de los 4 primeros bloques convolucionales, el quinto se re-e
# En base_model.layers.name tenemos la información del nombre de la capa
for layer in base_model.layers:
  if layer.name == 'block5_conv3':
    break
  layer.trainable = False
  print('Capa ' + layer.name + ' congelada...')

# Cogemos la última capa del model y le añadimos nuestro clasificador (top_model)
last = base_model.layers[-1].output #(X)
x = Flatten()(last) #(X)
x = Dense(1000, activation='relu', name='fc1')(x) #(X)
x= Dropout(0.3)(x) #(X)
x = Dense(200, activation='relu', name='fc2')(x) #(X)
x = Dense(10, activation='softmax', name='predictions')(x) #(X)
model = Model(base_model.input, x) #(X)

# Compilamos el modelo
optimizer=Adam(lr=0.01)
model.compile(optimizer=optimizer, loss='categorical_crossentropy', metrics=['accur
```

```python
# Vamos a visualizar el modelo prestando especial atención en el número de pesos to
# ¿tiene sentido en comparación al ejemplo de transfer learning?
model.summary()

# Entrenamos el modelo
H = model.fit(trainX, trainY, validation_split=0.2, batch_size=64, epochs=20, verbo

# Evaluación del modelo
print("[INFO]: Evaluando el modelo...")
predictions = model.predict(testX, batch_size=64)
# Obtener el report de clasificación
print(classification_report(testY.argmax(axis=1), predictions.argmax(axis=1), targe

# Gráficas
plt.style.use("ggplot")
plt.figure()
plt.plot(np.arange(0, 20), H.history["loss"], label="train_loss")
plt.plot(np.arange(0, 20), H.history["val_loss"], label="val_loss")
plt.plot(np.arange(0, 20), H.history["accuracy"], label="train_acc")
plt.plot(np.arange(0, 20), H.history["val_accuracy"], label="val_acc")
plt.title("Training Loss and Accuracy")
plt.xlabel("Epoch #")
plt.ylabel("Loss/Accuracy")
plt.legend()
plt.show()
```

Model: "vgg16"

Layer (type)	Output Shape	Param #
input_layer_7 (InputLayer)	(None, 32, 32, 3)	0
block1_conv1 (Conv2D)	(None, 32, 32, 64)	1,792
block1_conv2 (Conv2D)	(None, 32, 32, 64)	36,928
block1_pool (MaxPooling2D)	(None, 16, 16, 64)	0
block2_conv1 (Conv2D)	(None, 16, 16, 128)	73,856
block2_conv2 (Conv2D)	(None, 16, 16, 128)	147,584
block2_pool (MaxPooling2D)	(None, 8, 8, 128)	0
block3_conv1 (Conv2D)	(None, 8, 8, 256)	295,168
block3_conv2 (Conv2D)	(None, 8, 8, 256)	590,080
block3_conv3 (Conv2D)	(None, 8, 8, 256)	590,080
block3_pool (MaxPooling2D)	(None, 4, 4, 256)	0
block4_conv1 (Conv2D)	(None, 4, 4, 512)	1,180,160
block4_conv2 (Conv2D)	(None, 4, 4, 512)	2,359,808
block4_conv3 (Conv2D)	(None, 4, 4, 512)	2,359,808
block4_pool (MaxPooling2D)	(None, 2, 2, 512)	0
block5_conv1 (Conv2D)	(None, 2, 2, 512)	2,359,808
block5_conv2 (Conv2D)	(None, 2, 2, 512)	2,359,808
block5_conv3 (Conv2D)	(None, 2, 2, 512)	2,359,808
block5_pool (MaxPooling2D)	(None, 1, 1, 512)	0

Total params: 14,714,688 (56.13 MB)

Trainable params: 14,714,688 (56.13 MB)

Non-trainable params: 0 (0.00 B)

```
Capa input_layer_7 congelada...
Capa block1_conv1 congelada...
Capa block1_conv2 congelada...
Capa block1_pool congelada...
Capa block2_conv1 congelada...
Capa block2_conv2 congelada...
Capa block2_pool congelada...
Capa block3_conv1 congelada...
Capa block3_conv2 congelada...
Capa block3_conv3 congelada...
Capa block3_pool congelada...
Capa block4_conv1 congelada...
Capa block4_conv2 congelada...
Capa block4_conv3 congelada...
Capa block4_pool congelada...
Capa block5_conv1 congelada...
Capa block5_conv2 congelada...
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
/tmp/ipykernel_4256/3104871139.py in <cell line: 0>()
     51
     52 # Compilamos el modelo
---> 53 optimizer=Adam(lr=0.01)
     54 model.compile(optimizer=optimizer, loss='categorical_crossentropy', metrics=
['accuracy'])
     55

/usr/local/lib/python3.12/dist-packages/keras/src/optimizers/adam.py in __init__(sel
f, learning_rate, beta_1, beta_2, epsilon, amsgrad, weight_decay, clipnorm, clipvalu
e, global_clipnorm, use_ema, ema_momentum, ema_overwrite_frequency, loss_scale_facto
r, gradient_accumulation_steps, name, **kwargs)
     60         **kwargs,
     61     ):
---> 62         super().__init__(
     63             learning_rate=learning_rate,
     64             name=name,

/usr/local/lib/python3.12/dist-packages/keras/src/backend/tensorflow/optimizer.py in
__init__(self, *args, **kwargs)
     19 class TFOptimizer(KerasAutoTrackable, base_optimizer.BaseOptimizer):
     20     def __init__(self, *args, **kwargs):
---> 21         super().__init__(*args, **kwargs)
     22         self._distribution_strategy = tf.distribute.get_strategy()
     23

/usr/local/lib/python3.12/dist-packages/keras/src/optimizers/base_optimizer.py in __
init__(self, learning_rate, weight_decay, clipnorm, clipvalue, global_clipnorm, use_
ema, ema_momentum, ema_overwrite_frequency, loss_scale_factor, gradient_accumulation
_steps, name, **kwargs)
     88             )
     89         if kwargs:
---> 90             raise ValueError(f"Argument(s) not recognized: {kwargs}")
     91
     92         if name is None:

ValueError: Argument(s) not recognized: {'lr': 0.01}
```

EJERCICIO PROPUESTO 2: Combinar fine tuning con la técnica de data augmentation para reducir overfitting.

In [14]:
```
import tensorflow as tf

from tf.keras.preprocessing.image import ImageDataGenerator

datagen = ImageDataGenerator(
    rotation_range=15, # grados de rotacion aleatoria
    width_shift_range=0.2, # fraccion del total (1) para mover la imagen
    height_shift_range=0.2, # fraccion del total (1) para mover la imagen
    horizontal_flip=True, # girar las imagenes horizontalmente (eje vertical)
    # shear_range=0, # deslizamiento
    zoom_range=0.2, # rango de zoom
    # fill_mode='nearest', # como rellenar posibles nuevos pixeles
    # channel_shift_range=0.2 # cambios aleatorios en los canales de la imagen
    validation_split=0.2,
)
```

```
---------------------------------------------------------------------------
ModuleNotFoundError                       Traceback (most recent call last)
/tmp/ipykernel_4256/1241419969.py in <cell line: 0>()
      1 import tensorflow as tf
      2
----> 3 from tf.keras.preprocessing.image import ImageDataGenerator
      4
      5 datagen = ImageDataGenerator(

ModuleNotFoundError: No module named 'tf'

---------------------------------------------------------------------------
NOTE: If your import is failing due to a missing package, you can
manually install dependencies using either !pip or !apt.

To view examples of installing some common dependencies, click the
"Open Examples" button below.
---------------------------------------------------------------------------
```

In [15]:
```
# Imports que vamos a necesitar
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.optimizers import SGD, Adam
from tensorflow.keras.applications import VGG16, imagenet_utils
from tensorflow.keras.utils import to_categorical
from tensorflow.keras import optimizers
from tensorflow.keras.layers import Dropout, Flatten, GlobalMaxPooling2D, Dense
from tensorflow.keras import Model
import matplotlib.pyplot as plt
from sklearn.metrics import classification_report
import numpy as np
from sklearn.preprocessing import LabelBinarizer

#Cargamos el dataset CIFAR10
(trainX, trainY), (testX, testY) = cifar10.load_data()

# Normalizamos las entradas de idéntica forma a como lo hicieron para entrenar la V
```

```python
trainX = imagenet_utils.preprocess_input(trainX)
testX = imagenet_utils.preprocess_input(testX)

# Definimos dimensiones de nuestros datos de entrada y lista con las categorias de
input_shape = (32, 32, 3)
labelNames = ["Avión", "Automóvil", "Pájaro", "Gato", "Ciervo", "Perro", "Rana", "C

# En caso de inestabilidades numéricas pasar datos a one-hot encoding
lb = LabelBinarizer()
trainY = lb.fit_transform(trainY)
testY = lb.transform(testY)

# Importamos VGG16 con pesos de imagenet y sin top_model especificando tamaño de en
base_model = VGG16(weights='imagenet', include_top=False, input_shape=input_shape)
# Mostramos la arquitectura
base_model.summary()

# Congelamos las capas de los 4 primeros bloques convolucionales, el quinto se re-e
# En base_model.layers.name tenemos la información del nombre de la capa
for layer in base_model.layers:
  if layer.name == 'block5_conv3':
    break
  layer.trainable = False
  print('Capa ' + layer.name + ' congelada...')

# Cogemos la última capa del model y le añadimos nuestro clasificador (top_model)
last = base_model.layers[-1].output #(X)
x = Flatten()(last) #(X)
x = Dense(1000, activation='relu', name='fc1')(x) #(X)
x = Dropout(0.3)(x) #(X)
x = Dense(200, activation='relu', name='fc2')(x) #(X)
x = Dense(10, activation='softmax', name='predictions')(x) #(X)
model = Model(base_model.input, x) #(X)

# Compilamos el modelo
optimizer=Adam(lr=0.01)
model.compile(optimizer=optimizer, loss='categorical_crossentropy', metrics=['accur

# Vamos a visualizar el modelo prestando especial atención en el número de pesos to
# ¿tiene sentido en comparación al ejemplo de transfer learning?
model.summary()


# Entrenamiento de la red
print("[INFO]: Entrenando la red...")
H = model.fit(datagen.flow(trainX, trainY, batch_size=128, subset="training"), vali

# Evaluación del modelo
print("[INFO]: Evaluando el modelo...")
predictions = model.predict(testX, batch_size=64)
# Obtener el report de clasificación
print(classification_report(testY.argmax(axis=1), predictions.argmax(axis=1), targe

# Gráficas
plt.style.use("ggplot")
plt.figure()
```

```
plt.plot(np.arange(0, 20), H.history["loss"], label="train_loss")
plt.plot(np.arange(0, 20), H.history["val_loss"], label="val_loss")
plt.plot(np.arange(0, 20), H.history["accuracy"], label="train_acc")
plt.plot(np.arange(0, 20), H.history["val_accuracy"], label="val_acc")
plt.title("Training Loss and Accuracy")
plt.xlabel("Epoch #")
plt.ylabel("Loss/Accuracy")
plt.legend()
plt.show()
```

Model: "vgg16"

Layer (type)	Output Shape	Param #
input_layer_8 (InputLayer)	(None, 32, 32, 3)	0
block1_conv1 (Conv2D)	(None, 32, 32, 64)	1,792
block1_conv2 (Conv2D)	(None, 32, 32, 64)	36,928
block1_pool (MaxPooling2D)	(None, 16, 16, 64)	0
block2_conv1 (Conv2D)	(None, 16, 16, 128)	73,856
block2_conv2 (Conv2D)	(None, 16, 16, 128)	147,584
block2_pool (MaxPooling2D)	(None, 8, 8, 128)	0
block3_conv1 (Conv2D)	(None, 8, 8, 256)	295,168
block3_conv2 (Conv2D)	(None, 8, 8, 256)	590,080
block3_conv3 (Conv2D)	(None, 8, 8, 256)	590,080
block3_pool (MaxPooling2D)	(None, 4, 4, 256)	0
block4_conv1 (Conv2D)	(None, 4, 4, 512)	1,180,160
block4_conv2 (Conv2D)	(None, 4, 4, 512)	2,359,808
block4_conv3 (Conv2D)	(None, 4, 4, 512)	2,359,808
block4_pool (MaxPooling2D)	(None, 2, 2, 512)	0
block5_conv1 (Conv2D)	(None, 2, 2, 512)	2,359,808
block5_conv2 (Conv2D)	(None, 2, 2, 512)	2,359,808
block5_conv3 (Conv2D)	(None, 2, 2, 512)	2,359,808
block5_pool (MaxPooling2D)	(None, 1, 1, 512)	0

Total params: 14,714,688 (56.13 MB)

Trainable params: 14,714,688 (56.13 MB)

Non-trainable params: 0 (0.00 B)

```
Capa input_layer_8 congelada...
Capa block1_conv1 congelada...
Capa block1_conv2 congelada...
Capa block1_pool congelada...
Capa block2_conv1 congelada...
Capa block2_conv2 congelada...
Capa block2_pool congelada...
Capa block3_conv1 congelada...
Capa block3_conv2 congelada...
Capa block3_conv3 congelada...
Capa block3_pool congelada...
Capa block4_conv1 congelada...
Capa block4_conv2 congelada...
Capa block4_conv3 congelada...
Capa block4_pool congelada...
Capa block5_conv1 congelada...
Capa block5_conv2 congelada...
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
/tmp/ipykernel_4256/1478981322.py in <cell line: 0>()
     51
     52 # Compilamos el modelo
---> 53 optimizer=Adam(lr=0.01)
     54 model.compile(optimizer=optimizer, loss='categorical_crossentropy', metrics=
['accuracy'])
     55

/usr/local/lib/python3.12/dist-packages/keras/src/optimizers/adam.py in __init__(sel
f, learning_rate, beta_1, beta_2, epsilon, amsgrad, weight_decay, clipnorm, clipvalu
e, global_clipnorm, use_ema, ema_momentum, ema_overwrite_frequency, loss_scale_facto
r, gradient_accumulation_steps, name, **kwargs)
     60         **kwargs,
     61     ):
---> 62         super().__init__(
     63             learning_rate=learning_rate,
     64             name=name,

/usr/local/lib/python3.12/dist-packages/keras/src/backend/tensorflow/optimizer.py in
__init__(self, *args, **kwargs)
     19 class TFOptimizer(KerasAutoTrackable, base_optimizer.BaseOptimizer):
     20     def __init__(self, *args, **kwargs):
---> 21         super().__init__(*args, **kwargs)
     22         self._distribution_strategy = tf.distribute.get_strategy()
     23

/usr/local/lib/python3.12/dist-packages/keras/src/optimizers/base_optimizer.py in __
init__(self, learning_rate, weight_decay, clipnorm, clipvalue, global_clipnorm, use_
ema, ema_momentum, ema_overwrite_frequency, loss_scale_factor, gradient_accumulation
_steps, name, **kwargs)
     88             )
     89         if kwargs:
---> 90             raise ValueError(f"Argument(s) not recognized: {kwargs}")
     91
     92         if name is None:

ValueError: Argument(s) not recognized: {'lr': 0.01}
```

Perceptrón vs. Neurona Convolucional: Una Comparación Detallada

Perceptrón: La Base de las Redes Neuronales

El Perceptrón es el bloque de construcción fundamental de las redes neuronales artificiales, representando el modelo más simple de una neurona. Su funcionamiento se basa en la **recepción de múltiples entradas**, cada una ponderada por un **peso específico**, a las que se les suma un **sesgo (bias)**. El resultado de esta suma ponderada se somete a una **función de activación** para producir una única salida.

Matemáticamente, su operación se expresa como:

$$ y = f(\sum_{i=1}^{n} w_ix_i + b) $$

Donde:

- x_i son las entradas.
- w_i son los pesos asociados a cada entrada.
- b es el sesgo.
- f es la función de activación.

Propiedades Clave del Perceptrón:

- **Simplicidad:** Un modelo de una sola neurona, ideal para entender los principios básicos.
- **Clasificación Lineal:** Es inherentemente apto para resolver problemas de clasificación linealmente separables.
- **Aplicaciones:** Principalmente utilizado con datos tabulares o numéricos, como la clasificación de spam o predicciones binarias simples.

Neurona Convolucional (CNN): Especialización en Datos Estructurados

A diferencia del perceptrón, la Neurona Convolucional está diseñada específicamente para procesar datos con una estructura espacial, siendo su aplicación más destacada en el **procesamiento de imágenes**. En lugar de operar sobre entradas planas, una neurona convolucional aplica un **filtro (kernel)** que se desliza por la imagen (o datos estructurados), realizando operaciones de convolución. Este proceso permite la detección automática y jerárquica de características visuales.

La operación fundamental se puede describir como:

$$ FeatureMap = Imagen * Filtro $$

Este mecanismo es crucial para identificar elementos como:

- Bordes y líneas.
- Texturas y gradientes.
- Formas y contornos.
- Objetos complejos.

Propiedades Clave de la Neurona Convolucional:

- **Análisis Espacial:** Óptima para trabajar con imágenes, video, audio y otros datos estructurados.

- **Extracción de Características Automática:** Utiliza filtros para aprender y extraer características relevantes de los datos de entrada.
- **Arquitectura Compleja:** Formando parte de Redes Neuronales Convolucionales (CNNs), permite construir modelos profundos y altamente eficientes para tareas visuales.

Diferencias Fundamentales y Ámbitos de Aplicación

Característica Principal	Perceptrón	Neurona Convolucional
Naturaleza del Modelo	Red neuronal fundamental y simple	Componente clave de redes neuronales profundas (CNN)
Tipo de Entrada	Datos numéricos genéricos, vectorizados	Imágenes, matrices de datos espaciales
Mecanismo Operativo	Suma ponderada de entradas y activación	Operación de convolución con filtros
Complejidad del Problema	Principalmente problemas de clasificación binaria y lineal	Complejos problemas de visión por computador
Rol Principal	Aprendizaje de patrones simples	Detección jerárquica de características y patrones espaciales

Ejemplos de Uso Práctico:

Perceptrón:

- Detección de spam en correos electrónicos (clasificación binaria).
- Sistemas de recomendación simples basados en preferencias numéricas.
- Predicción de resultados simples en base a un conjunto limitado de características.

Neurona Convolucional (dentro de una CNN):

- Reconocimiento de objetos en imágenes y videos (p.ej., identificar coches, personas).
- Diagnóstico médico a partir de imágenes (p.ej., detección de tumores en radiografías).
- Reconocimiento facial y biometría.
- Conducción autónoma (interpretación del entorno visual).

4. Github

https://github.com/Erick-305/machine-learning.git